Knowledge Tracing for Large Language Models

Project Workbook 2: Project Design and Early Implementation

By

Ronak Patel
Bhagyesh Rathi
Aniket Sarrin

October 29, 2026

Advisor: Dr. Mahima Suresh Agumbe

CMPE 295A, San José State University

Chapter 1
Literature Search, State of the Art

1.1 Literature Search

This project sits at the intersection of four literatures that have not previously been combined: knowledge tracing from educational data mining, psychometric evaluation of language models, the study of LLM training dynamics across checkpoints, and temporal item response theory from political science and learning analytics. The search covered arXiv (cs.CL, cs.LG, cs.AI), the ACL Anthology, OpenReview (ICLR 2025 and 2026, NeurIPS 2024 and 2025, ICML 2026), the EDM/ AIED/LAK proceedings, Semantic Scholar forward citations of the five closest papers, and GitHub and HuggingFace for released data and code. Roughly 60 queries and 80 forward citations were screened; about 45 papers were read in full. The search was carried out in three passes (September 2–3, September 22 and September 30, 2026): the first pass fact-checked each recent claim against primary sources and produced the initial reference list and a first project plan, and the third pass was spot-verified against primary sources. Cross-references below are to the numbered entries in the References section.

Knowledge tracing

Knowledge tracing (KT) is the learner model inside intelligent tutoring systems. Bayesian Knowledge Tracing [1] models each skill as a two-state hidden Markov model (unmastered/mastered) with per-skill initial-mastery, learn, guess and slip probabilities, updated after every observed response. Khajah, Lindsey and Mozer [2] showed that adding forgetting to BKT removes about half of DKT's reported advantage on Assistments, and that other extensions (per-student ability, skill discovery) close more of it, which matters for us because our data has few learners. Deep Knowledge Tracing [3] replaced the HMM with a recurrent network, and later attention-based models such as AKT [4] and SAINT [45] added item and context representations; these are trained on large educational datasets with thousands of student sequences, and their behaviour with a handful of learners is not established. Yeung and Yeung [49] analysed consistency problems in DKT's predictions, and Vie and Kashima [5] showed that DKT is an implicit dynamic multidimensional IRT model, which gives the bridge between KT and item response theory that our method relies on. pyBKT [6] is the maintained open-source BKT implementation; its internal data format allows several observations per time step with a single transition per step, but its E-step multiplies raw emission probabilities across observations before normalising, which underflows for batches of hundreds of items, so a log-space implementation is needed for our data. DAS3H [7] is a learned non-latent baseline that combines item difficulty, skill and

time-window counts in a logistic model; its practice-window features must be re-defined over training progress for our setting.

Psychometric evaluation of language models

Item response theory (IRT) treats each benchmark question as an item with a difficulty and a discrimination and each model as a respondent with a latent ability. Fluid Benchmarking [8] from AI2 fits 2PL IRT to Open LLM Leaderboard results and then selects items adaptively as a model improves during pretraining; its paper reports 467 checkpoints of six open models, and the public release we inspected contains 452 of them with the per-item, per-checkpoint correctness that forms our Phase 1 dataset. Item Response Scaling Laws [9] fits IRT to 6,612 pretraining checkpoints and 37,682 items from the DataDecide release [44], estimates each checkpoint's ability independently by adaptive testing, and fits a scaling law in log-compute through those estimates. It is the closest prior work on the psychometric side. It has no sequential state linking checkpoints, its item parameters are calibrated once and frozen, it has no per-skill decomposition, and it validates across model sizes and item subsets rather than by forecasting later checkpoints within a run; forgetting is not something its scaling law represents. Jiang et al. [10] study when IRT estimates for AI evaluation can be trusted: in their simulations 30 respondents were insufficient for item recovery and 100 were adequate (sizes in between were not tested), and rank recovery degraded sharply under skewed ability distributions, which a pretraining run has by construction. Zheng and Yang [11] show differential item functioning across model families on MMLU, HellaSwag and WinoGrande. Paquin and Jain [12] audit MMLU with a 2PL model over 1,000 models and find a latent correlation of 0.965 between STEM and non-STEM ability; they conclude the two are distinguishable (a two-dimensional model fits held-out responses better), which motivates testing how separable the 57 subjects are rather than assuming either answer. Beyond Scores [13] fits multidimensional IRT with a 35-dimension mathematics Q-matrix to 41 frozen models, and Laplace-PSN-IRT [14] puts an approximately calibrated posterior on LLM ability while its joint item posterior is poorly calibrated; both are static. Cognitive diagnosis has been applied to 30 frozen financial LLMs [15], and CDS [16] uses a per-skill diagnosis to synthesise training data for weak skills, which is the downstream application of our mastery estimates. ATLAS [17] releases calibrated item banks (primarily 3PL, with a 2PL comparison) over 3,000 models for five benchmarks, not including MMLU, which we can use to cross-check Fluid's calibration where items overlap. Signal and Noise [18] quantifies checkpoint-to-checkpoint evaluation noise on our exact benchmarks and shows that averaging intermediate checkpoints and filtering low-signal MMLU subtasks improve decision accuracy; it is the aggregate-level precedent that motivates our item-history baseline, though that baseline is inspired by rather than an implementation of their method.

LLM training dynamics across checkpoints

Pythia [19] and OLMo [20] made intermediate pretraining checkpoints public and started a literature on what models learn when. Mishra, Poesia and Goodman [21] introduced MathCAMPS, a synthetic math benchmark tagged with 44 Common Core standards, and showed that Pythia, OLMo and Amber acquire math skills in roughly the order schools teach them; they report only per-skill accuracy curves. Liu et al. [22] tracked 91 elemental and composite tasks across nine models and found that emergence order (defined by an accuracy threshold) is stable across models and that composite skills usually appear after their components; they also predict held-out composite-task trajectories from function vectors, and their repository releases per-item correctness for Amber, OLMo-2 and Pythia-6.9B. Chang et al. [23] injected fictional facts into OLMo pretraining and observed forgetting that follows a power law in training steps, which motivates testing non-constant or time-scaled forgetting in our models. Harmon et al. [24] count item-level correct-to-incorrect transitions to map forgetting during post-training, and Liao et al. [25] study entity forgetting during pretraining. Hu et al. [26] fit hidden Markov models to aggregate training metrics to find latent phases of training, so latent-state models of training do exist at the aggregate level; Michaelov, Levy and Bergen [27] show that behavioural phases are consistent across architectures. Dominguez-Olmedo, Dorner and Hardt [28] show that, under their evaluation, Pythia performs at chance on MMLU and that apparent differences between model families shrink after equal fine-tuning on task-relevant data, which we must address when interpreting our cross-model results. Among these, none fits a per-skill latent learner state to item-level checkpoint responses and evaluates it by forecasting item-level responses at later checkpoints within a training run: Hu et al. model aggregate metrics, and Liu et al. forecast task-level trajectories.

Temporal item response theory

Dynamic IRT models let ability evolve over time. Martin and Quinn [29] introduced a dynamic ideal-point model for the U.S. Supreme Court (29 justices, at most nine sitting at once, over 47 terms, with different cases each term), with a random-walk prior on ability estimated by MCMC using Kalman forward-filtering and backward-sampling. Imai, Lo and Olmsted [30] provide a fast variational EM implementation in the R package emIRT (function dynIRT). This is the closest structural precedent for few learners observed over many periods, but it is not a drop-in fit: dynIRT assigns each item to exactly one period, so sharing an item's difficulty and discrimination across all checkpoints (our setting, where the same 27,842 items are re-asked at every checkpoint) requires adapting the model, and its manual states that its variational variance estimates are far too small to use as uncertainty. T-SKIRT [31] from Knewton models ability as a Wiener process indexed by elapsed time with a

closed-form effective discrimination that down-weights old responses; its temporal-only ablation gains 2.2 accuracy points and 1.7 AUC points over static IRT, and the full model with structured multidimensional proficiencies gains 2.8 and 2.4. Its closed form is derived for a probit link, so logistic item parameters need conversion. VTIRT [32] provides amortised variational inference for dynamic IRT but assumes one item per time step and was evaluated with thousands of learners; its behaviour with a few long trajectories is untested. GD-GPIRT [33] uses Gaussian processes to let both ability and item response functions drift, at high computational cost, and validates by forecasting several periods ahead. Yan, Tang and Shimada [34] correct per-item bias in deployed knowledge-tracing models with empirical-Bayes shrinkage under a state-space model and derive an information floor that explains why temporal tracking of per-item parameters gives no benefit at the data densities of four standard KT datasets; a first-order application of that bound to our data density gives about 2.3 logits per item per checkpoint, which means per-item drift tests must be re-posed at the item-bank level or in coarse time bins, and a project-specific simulation-based power analysis is needed because the bound assumes independent observations. Wang, Berger and Burdick [35] is a foundational Bayesian treatment of dynamic IRT supporting both retrospective and online inference.

LLMs used for knowledge tracing of human students

A large recent literature uses LLMs to trace human students: LLM-KT [36] aligns an LLM with a KT objective, NTKT [37] reframes KT as next-token prediction, CLST [38] addresses cold start, and Scarlatos et al. [39] trace knowledge in tutor-student dialogues. These are the reverse of our direction (the LLM is the tracer, not the learner) and are cited to prevent confusion; our title and introduction state the direction explicitly.

Visualization of training and evaluation

DeepTracker [40] visualised a CNN's per-validation-image correctness across training iterations as a pixel chart with class-level rollups; it is the closest visual precedent for our dashboard. Zeno [41] offers per-slice sparklines across model versions and accepts lm-evaluation-harness output, so it can approximate per-skill curves over checkpoints without a learner model. LLM Comparator [42] shows side-by-side prompt-level comparisons of two models. The PSLC DataShop learning-curve report [43] is the display grammar used in education for per-knowledge-component curves. No existing tool shows a fitted per-skill mastery trajectory with uncertainty across training checkpoints.

1.2 State-of-the-Art Summary

For estimating what an LLM knows, the state of the art is static psychometrics. Fluid Benchmarking [8] fits 2PL IRT to leaderboard results, releases item parameters, and selects items adaptively during pretraining; Item Response Scaling Laws [9] scales the same idea to 6,612 checkpoints and folds ability into a scaling law; ATLAS [17] provides calibrated item banks; and Beyond Scores [13] adds a skill dimension through a Q-matrix. All of these estimate a fixed ability (or profile) for each frozen model or checkpoint independently. None maintains a state that carries information from one checkpoint to the next, none models forgetting, and none is validated by predicting the responses of a later checkpoint.

For describing how skills develop during training, the state of the art is accuracy curves with thresholds. MathCAMPS [21] and the Implicit Curriculum paper [22] show that skills emerge in a stable, curriculum-like order; Chang et al. [23] show power-law forgetting of injected facts; Harmon et al. [24] count item-level forgetting after post-training; Signal and Noise [18] quantifies how noisy a single checkpoint's score is and shows that averaging checkpoints helps. These works observe the phenomena we care about (emergence, forgetting, noise) without a per-skill latent learner state: Liu et al. do forecast held-out task-level trajectories, and Hu et al. [26] fit latent states to aggregate metrics, but none forecasts item-level responses at later checkpoints from a per-skill state with a transition kernel.

For learner modelling itself, the state of the art in education is deep KT (AKT [4], SAINT [45]) trained on large student populations, with BKT variants that include forgetting [2] remaining competitive and far more interpretable. For few learners observed over many periods, the closest precedents are random-walk dynamic IRT as in Martin and Quinn [29] with the emIRT implementation [30], which needs adaptation to share item parameters across periods, and T-SKIRT [31] as a cheap closed-form temporal correction to static IRT. The pyBKT library [6] is the maintained BKT implementation; batching hundreds of observations per step in it requires a log-space E-step.

On the industry side, the tools and data this project builds on come from applied groups: AI2 released Fluid Benchmarking [8], the OLMo checkpoints [20, 47] and DataDecide [44]; LLM360 and EleutherAI released the Amber, K2 and Pythia [19] checkpoint suites; Knewton developed T-SKIRT [31] for a production adaptive-learning system; Google released LLM Comparator [42]; and Zeno [41] is the behavioural-evaluation tool practitioners use for slice-level model comparison. None of these industry tools or releases fits a learner model over checkpoints.

Our project combines these: it treats each checkpoint as one moment in a learner's history, fits sequential learner models (BKT with forgetting, a Martin-Quinn-style dynamic IRT with tied item parameters, T-SKIRT, and deep KT as an ablation) to item-level checkpoint responses, validates them by forecasting held-out later checkpoints at the item level, tests whether item parameters stay stable across a single model's training run, and presents the fitted trajectories in an interactive tool. In the sources reviewed through September 30, 2026, we did not identify a method that jointly models a stochastic per-skill latent state across an LLM's pretraining checkpoints, treats each checkpoint's responses as a batch, and evaluates item-level predictions on later held-out checkpoints with calibrated uncertainty. Related work already provides checkpoint psychometrics [8, 9], scaling forecasts [9], task-level emergence prediction [22], aggregate latent training phases [26], and static skill diagnosis [13, 15, 16]. Our study will test whether sequential state modelling adds predictive and diagnostic value beyond these.

On the data side, the state of the art has moved quickly. Fluid Benchmarking's release (six models, 452 checkpoints, 27,842 usable items) is in hand and ingested. DataDecide-eval-instances [44] (about 150,000 checkpoints of models up to 1B parameters and 500 million item-level evaluations according to its dataset card, ODC-BY) and the Implicit Curriculum repository [22] (per-item results for Amber, OLMo-2 and Pythia-6.9B on 91 skill-structured tasks, MIT) are public, and we found no learner model fitted to either. Phase 1 therefore needs no GPU time. The preferred Phase 2 extension (OLMo-2 on MathCAMPS) does need GPU access; if it cannot be secured, these released datasets provide a no-inference fallback with a different skill taxonomy.

1.3 References

[1] Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4(4), 253–278.

The original Bayesian Knowledge Tracing paper. Defines the two-state HMM per skill with learn, guess and slip parameters that our BKT branch uses.

[2] Khajah, M., Lindsey, R. V., & Mozer, M. C. (2016). How deep is knowledge tracing? Proceedings of the 9th International Conference on Educational Data Mining. arXiv:1604.02416.

Shows that BKT extended with forgetting, ability and skill discovery recovers about half of DKT's advantage. Motivates BKT with forgetting as a primary model given our small number of learners.

[3] Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. Advances in Neural Information Processing Systems, 28.

Introduces LSTM-based knowledge tracing. Our deep KT ablation follows this family; the paper also establishes the next-response prediction protocol we adopt.

[4] Ghosh, A., Heffernan, N., & Lan, A. S. (2020). Context-aware attentive knowledge tracing. Proceedings of KDD 2020. arXiv:2007.12324.

Attention-based KT with item difficulty embedding. Representative of modern deep KT, developed and evaluated on large student populations; its behaviour with a handful of learners is not established.

[5] Vie, J.-J., & Kashima, H. (2023). Deep knowledge tracing is an implicit dynamic multidimensional item response theory model. Proceedings of ICCE 2023. arXiv:2309.12334.

Theoretical bridge between KT and IRT. Justifies treating BKT and temporal IRT as two members of one model family in our design.

[6] Badrinath, A., Wang, F., & Pardos, Z. (2021). pyBKT: An accessible Python library of Bayesian knowledge tracing models. Proceedings of EDM 2021. arXiv: 2105.00385.

Maintained MIT-licensed BKT implementation with forgetting, per-item guess/slip and multiple learn rates. Its internal data format supports batched observations per time step, which we rely on.

[7] Choffin, B., Popineau, F., Bourda, Y., & Vie, J.-J. (2019). DAS3H: Modeling student learning and forgetting for optimally scheduling distributed practice of skills. Proceedings of EDM 2019. arXiv:1905.06873.

Logistic model over item, skill and time-window features. Adopted as a strong non-latent baseline.

[8] Hofmann, V., Heineman, D., Magnusson, I., Lo, K., Dodge, J., Sap, M., Koh, P. W., Wang, C., Hajishirzi, H., & Smith, N. A. (2025). Fluid language model benchmarking. Proceedings of COLM 2025. arXiv:2509.11106.

Static 2PL IRT plus adaptive item selection over pretraining checkpoints. The paper evaluates 467 checkpoints of six models; the public release we ingested contains 452. Source of our Phase 1 dataset and our static-IRT baseline; also reports that random-subset accuracy is non-monotonic late in training, a competing explanation for forgetting that we must rule out.

[9] Truong, S., Tu, Y., Schaeffer, R., & Koyejo, S. (2026). Item response scaling laws: A measurement theory approach for efficient and generalizable neural scaling estimation. arXiv:2606.07616.

Fits IRT to 6,612 pretraining checkpoints, estimating each checkpoint's ability independently and then fitting a log-compute scaling law through the estimates. The closest psychometric prior work; no sequential state, frozen item parameters, no per-skill decomposition, and validation across sizes and item subsets rather than by forecasting later checkpoints within a run.

[10] Jiang, H., Kwon, S., Luo, J., Xiao, Z., & Zhang, S. (2026). Can we trust item response theory for AI evaluation? arXiv:2607.15190.

Simulation study of IRT estimator reliability. Finds N = 30 respondents insufficient and N = 100 adequate for item recovery in its conditions (sizes between were not tested), that skewed ability distributions harm ranking recovery, and recommends a parametric-recovery simulation with one's own fitted parameters, which we adopt as a validation step.

[11] Zheng, Q., & Yang, Y. (2026). Are near-tied LLM rankings robust to family-DIF-guided benchmark recomposition? arXiv:2609.00482.

Differential item functioning across model families on MMLU, HellaSwag and WinoGrande. Establishes item non-invariance across populations; our test is within one model across its checkpoints.

[12] Paquin, D., & Jain, R. (2026). What does MMLU actually measure? A psychometric audit of difficulty structure in aggregate benchmark scores. arXiv: 2609.09372.

2PL audit of all 14,042 MMLU items over 1,000 models. Reports a 0.965 latent correlation between STEM and non-STEM ability, concludes the two are distinguishable (a two-dimensional model fits held-out responses better), and recommends clustering standard errors by subject; both shape our MMLU-subject analysis.

[13] Zhang, X., Gong, X., Qin, J., Wang, Q., Liao, J., Wang, Z., Feng, D., & Ding, B. (2026). Beyond scores: Diagnostic LLM evaluation via fine-grained abilities. arXiv:2604.12191.

Multidimensional IRT with a 35-dimension mathematics Q-matrix over 41 frozen models, validated by held-out item prediction. Static per-skill profiling; we cite it to position per-skill trajectories as the new part. Its AUC is not comparable to ours because its held-out items use default item parameters.

[14] Mandujano Reyes, J. F. (2026). Laplace-PSN-IRT: Uncertainty quantification for neural item response theory models of LLM benchmarks. arXiv:2607.25257.

Approximately calibrated posterior over LLM ability via a Laplace approximation; the item branch is poorly calibrated when discrimination and difficulty are sampled jointly. Bounds what we may claim about uncertainty.

[15] Kuang, Z., Zhu, F., Jiang, M., Lai, Y., Wang, Z., Wang, Z., Qiu, M., Huang, J., Peng, M., Xie, Q., & Ananiadou, S. (2025). From scores to skills: A cognitive

diagnosis framework for evaluating financial large language models. arXiv: 2508.13491.

Cognitive diagnosis of 30 frozen financial LLMs at the skill level. Static analogue of our per-skill profile.

[16] Zhao, H., Han, J., Liang, J., Xiao, Y., Meng, X., & Wei, J. (2025). CDS: Knowledge component-driven data synthesis guided by cognitive diagnosis theory. arXiv:2501.07674.

Uses per-skill cognitive diagnosis of an LLM to synthesise training data for weak skills. The downstream application our mastery estimates would feed.

[17] Li, P., et al. (2025). Adaptive testing for LLM evaluation: A psychometric alternative to static benchmarks. arXiv:2511.04689 (v3, ICML 2026 spotlight).

Releases calibrated item banks (primarily 3PL, with a 2PL comparison) over 3,000 models for five benchmarks; MMLU is not among them. Used to cross-check Fluid Benchmarking's calibration where items overlap.

[18] Heineman, D., Hofmann, V., Magnusson, I., Gu, Y., Smith, N. A., Hajishirzi, H., Lo, K., & Dodge, J. (2025). Signal and noise: A framework for reducing uncertainty in language model evaluation. Advances in Neural Information Processing Systems, 38. arXiv:2508.13144.

Defines checkpoint-to-checkpoint noise, shows averaging checkpoints and filtering low-SNR MMLU subtasks improve decision accuracy. The aggregate-level precedent for our smoothing baseline.

[19] Biderman, S., et al. (2023). Pythia: A suite for analyzing large language models across training and scaling. Proceedings of ICML 2023. arXiv:2304.01373.

Sixteen models (at eight parameter sizes, with deduplicated-data variants) with 154 public checkpoints each and identical data order. Two Pythia models are in our Phase 1 data; the full suite is a Phase 2 option.

[20] Groeneveld, D., et al. (2024). OLMo: Accelerating the science of language models. Proceedings of ACL 2024. arXiv:2402.00838.

Fully open model family with hundreds of released checkpoints. OLMo-1 7B is in our Phase 1 data. See [47] for OLMo 2.

[21] Mishra, S., Poesia, G., & Goodman, N. D. (2025). From next-token to mathematics: The learning dynamics of mathematical reasoning in language models. Proceedings of COLM 2025. arXiv:2407.00900.

MathCAMPS: 44 Common Core standards, skills emerge in curriculum order across Pythia, OLMo-2 and Amber checkpoints; also analyses post-training effects and robustness to follow-up questions. Its pretraining analysis reports per-skill accuracy curves without a sequential learner model. Our Phase 2 dataset and the per-skill accuracy-curve baseline we must beat.

[22] Liu, E., Sun, K., Li, M., Lee, I., Tjuatja, L., Huang, J., & Neubig, G. (2026). What do language models learn and when? The implicit curriculum hypothesis. arXiv: 2604.08510.

Threshold-based emergence order across 91 tasks and nine models; composite skills usually follow their components; predicts held-out composite-task trajectories from function vectors. Its repository releases per-item results for three of our learners on a skill-structured suite.

[23] Chang, H., Park, J., Ye, S., Yang, S., Seo, Y., Chang, D.-S., & Seo, M. (2024). How do large language models acquire factual knowledge during pretraining? Advances in Neural Information Processing Systems, 37. arXiv:2406.11813.

Injects fictional facts into OLMo pretraining and finds power-law forgetting of the injected knowledge. Motivates testing non-constant or time-scaled forgetting in our models; a different observable from ours, so not a direct prior.

[24] Harmon, J., Hochlehnert, A., Bethge, M., & Prabhu, A. (2025). Mapping post-training forgetting in language models at scale. arXiv:2510.17776.

Item-level correct-to-incorrect transition counts across about 100 subdomains after post-training. The counting baseline for forgetting.

[25] Liao, C., Xie, R., Sun, X., Sun, H., & Kang, Z. (2025). Exploring forgetting in large language model pre-training. Proceedings of ACL 2025. arXiv:2410.17018.

Entity-level forgetting during pretraining with new retention metrics. Related evidence that forgetting during pretraining is real.

[26] Hu, M. Y., Chen, A., Saphra, N., & Cho, K. (2023). Latent state models of training dynamics. Transactions on Machine Learning Research. arXiv:2308.09543.

Fits HMMs to aggregate training metrics to find latent phases. Methodological precedent for a latent-state view of training, at the aggregate rather than skill level.

[27] Michaelov, J. A., Levy, R. P., & Bergen, B. K. (2025). Language model behavioral phases are consistent across architecture, training data, and scale. Advances in Neural Information Processing Systems, 38. arXiv:2510.24963.

Shows structured behavioural phases across 1,400 checkpoints. Motivation that pretraining has phases worth modelling.

[28] Dominguez-Olmedo, R., Dorner, F. E., & Hardt, M. (2025). Training on the test task confounds evaluation and emergence. Proceedings of ICLR 2025. arXiv: 2407.07890.

Shows Pythia sits at chance on MMLU and that family differences shrink after equal fine-tuning on task data. A competing explanation for our cross-model tiers that we address in the discussion.

[29] Martin, A. D., & Quinn, K. M. (2002). Dynamic ideal point estimation via Markov chain Monte Carlo for the U.S. Supreme Court, 1953–1999. Political Analysis, 10(2), 134–153.

Dynamic ideal-point model for 29 Supreme Court justices over 47 terms with a random-walk prior on ability, estimated by MCMC with Kalman forward-filtering and backward-sampling; cases differ across terms. Structural precedent for our primary temporal-IRT model, which additionally needs item parameters tied across periods.

[30] Imai, K., Lo, J., & Olmsted, J. (2016). Fast estimation of ideal points with massive data. American Political Science Review, 110(4), 631–656.

Variational EM implementation of the Martin-Quinn model in the R package emIRT (function dynIRT, version 0.0.15, GPL-3). Each item is assigned to one period and the manual warns that the variational variances are unusable as uncertainty; we use it as a reference implementation and starting point, not a drop-in.

[31] Ekanadham, C., & Karklin, Y. (2017). T-SKIRT: Online estimation of student proficiency in an adaptive learning system. arXiv:1702.04282.

Temporal IRT with Wiener-process ability and closed-form time-decayed discrimination under a probit link; temporal-only ablation gains +2.2 accuracy / +1.7 AUC over static IRT, full model +2.8 / +2.4. Cheap first temporal model to run on our data, after converting Fluid's logistic item parameters.

[32] Kim, Y., Sankaranarayanan, S., Piech, C., & Thille, C. (2023). Variational temporal IRT: Fast, accurate, and explainable inference of dynamic learner proficiency. Proceedings of EDM 2023. arXiv:2311.08594.

Amortised variational dynamic IRT. Assumes one item per time step and was evaluated with thousands of learners; its recognition network is trained across learners, so its behaviour with six long trajectories is untested. Not adopted; cited as the fast-inference state of the art.

[33] Chen, Y., Montgomery, J. M., & Garnett, R. (2025). A dynamic, ordinal Gaussian process item response theoretic model. arXiv:2504.02643.

Gaussian-process dynamic IRT with time-varying item parameters and a forecast-ahead validation protocol. Used on subsamples for the item-drift question.

[34] Yan, X., Tang, C., & Shimada, A. (2026). Recovering stranded discrimination in knowledge tracing: Per-item bias correction via empirical-Bayes shrinkage. arXiv:2606.14123.

Empirical-Bayes shrinkage of per-item bias under a state-space model, with an information floor showing why per-item temporal tracking does not help at standard KT data densities; we use that bound to design our drift test.

[35] Wang, X., Berger, J. O., & Burdick, D. S. (2013). Bayesian analysis of dynamic item response models in educational testing. Annals of Applied Statistics, 7(1), 126–153.

Canonical Bayesian dynamic IRT reference. Background for the full-posterior refit of our final models.

[36] Wang, Z., et al. (2025). LLM-KT: Aligning large language models with knowledge tracing using a plug-and-play instruction. arXiv:2502.02945.

An LLM used to trace human students. Reverse direction from ours; cited to prevent confusion.

[37] Norris, M., Gal, K., & Bulathwela, S. (2025). Next token knowledge tracing: Exploiting pretrained LLM representations to decode student behaviour. arXiv: 2511.02599.

Reframes KT as next-token prediction over student histories. Reverse direction from ours.

[38] Jung, H., et al. (2024). CLST: Cold-start mitigation in knowledge tracing by aligning a generative language model as a students' knowledge tracer. arXiv: 2406.10296.

LLM as a tracer of human students under cold start. Reverse direction from ours.

[39] Scarlatos, A., Baker, R. S., & Lan, A. (2025). Exploring knowledge tracing in tutor-student dialogues using LLMs. Proceedings of LAK 2025. arXiv: 2409.16490.

Applies KT to dialogue turns with LLM-identified knowledge components. Reverse direction; also a precedent for KT on unconventional time steps.

[40] Liu, D., Cui, W., Jin, K., Guo, Y., & Qu, H. (2018). DeepTracker: Visualizing the training process of convolutional neural networks. ACM Transactions on Intelligent Systems and Technology, 10(1). arXiv:1808.08531.

Per-instance correctness across training iterations as a pixel chart with class rollups. The closest visual precedent for our dashboard.

[41] Cabrera, Á. A., Fu, E., Bertucci, D., Holstein, K., Talwalkar, A., Hong, J. I., & Perer, A. (2023). Zeno: An interactive framework for behavioral evaluation of machine learning. Proceedings of CHI 2023. arXiv:2302.04732.

Slice-based evaluation with per-slice sparklines across model versions. The practical substitute a reviewer would name; we position our tool against it.

[42] Kahng, M., Tenney, I., Pushkarna, M., Liu, M. X., Wexler, J., Reif, E., Kallarackal, K., Chang, M., Terry, M., & Dixon, L. (2024). LLM Comparator:

Visual analytics for side-by-side evaluation of large language models. CHI 2024 Extended Abstracts. arXiv:2402.10524.

Side-by-side model comparison by prompt category. Shows skill-grouped views exist; skill-grouped views over training time do not.

[43] Koedinger, K. R., Baker, R. S., Cunningham, K., Skogsholm, A., Leber, B., & Stamper, J. (2010). A data repository for the EDM community: The PSLC DataShop. In Handbook of Educational Data Mining (pp. 43–56). CRC Press.

Learning-curve reports per knowledge component. The display grammar we transfer to LLM checkpoints.

[44] Magnusson, I., et al. (2025). DataDecide: How to predict best pretraining data with small experiments. arXiv:2504.11393.

Its dataset card describes about 150,000 checkpoints of models up to 1B parameters with 500 million item-level evaluations (ODC-BY); we have verified the card and file layout, not recounted the totals. A no-GPU Phase 2 data option.

[45] Choi, Y., Lee, Y., Cho, J., Baek, J., Kim, B., Cha, Y., Shin, D., Bae, C., & Heo, J. (2020). Towards an appropriate query, key, and value computation for knowledge tracing. Proceedings of L@S 2020. arXiv:2002.07033.

SAINT: transformer-based knowledge tracing. Representative of deep KT that needs large student populations.

[46] Chen, M., Roberts, N., Bhatia, K., Wang, J., Zhang, C., Sala, F., & Ré, C. (2023). Skill-it! A data-driven skills framework for understanding and training language models. Advances in Neural Information Processing Systems, 36. arXiv: 2307.14430.

Online data selection that orders training data by skill prerequisites. Together with CDS [16], the kind of training-time use a per-skill mastery estimate would feed.

[47] OLMo Team. (2025). 2 OLMo 2 Furious. arXiv:2501.00656.

OLMo 2 technical report. OLMo-2 7B is in our Phase 1 data and is the Phase 2 target; its released intermediate checkpoints and training artifacts are what "public training history" refers to.

[48] Kübler, J., Budhathoki, K., Kleindessner, M., Zhou, X., Yin, J., Khetan, A., & Karypis, G. (2026). When LLMs get significantly worse: A statistical approach to detect model degradations. Proceedings of ICLR 2026. arXiv:2602.10144.

Exact one-sided McNemar test on paired item responses with pooled, max-drop and Fisher aggregation across tasks. The paired test we will use for forgetting; selection of the peak across checkpoints and screening across subjects must be built into the test family or a null simulation.

[49] Yeung, C.-K., & Yeung, D.-Y. (2018). Addressing two problems in deep knowledge tracing via prediction-consistent regularization. Proceedings of L@S 2018. arXiv:1806.02180.

Identifies reconstruction and inconsistency problems in DKT's predictions. Cited for the known weaknesses of deep KT outputs as mastery estimates.

Chapter 2
Project Justification

Large language models are evaluated with a single benchmark score per checkpoint. That number cannot say which skills a model has acquired, when each one appeared, whether any were lost later in training, or how confident anyone should be in the estimate. Developers who run pretraining, and researchers who study it, currently answer those questions by eye from accuracy curves. Intelligent tutoring systems addressed the same problem for human students thirty years ago with knowledge tracing, which maintains a per-skill mastery estimate, updates it after each response, and is validated by predicting the learner's next response. The literature search in Chapter 1 (three passes, September 2–3, 22 and 30, 2026, the first fact-checked against primary sources, the third spot-verified against primary sources and then independently reviewed) found that the closest prior work either fits a static ability to each checkpoint independently (Item Response Scaling Laws [9], Fluid Benchmarking [8]), describes emergence and forgetting with accuracy thresholds and transition counts (MathCAMPS [21], Implicit Curriculum [22], Harmon et al. [24]), or models latent phases of training at the aggregate level (Hu et al. [26]). In the sources reviewed, we did not identify a method that maintains a stochastic per-skill state with a transition kernel over checkpoints, treats each checkpoint's responses as a batch, forecasts item-level responses at later held-out checkpoints with calibrated uncertainty, and tests whether item parameters stay stable across a single model's run. A search cannot prove that no such method exists anywhere; it establishes that the question is open in the literature we can find.

The project is feasible within two semesters because the hardest dependency is already resolved. Fluid Benchmarking's release provides per-item, per-checkpoint correctness for six open models across 452 checkpoints and 27,842 items, already ingested into a canonical table of 12.6 million binary responses, so Phase 1 needs no GPU. The preferred Phase 2 extension (OLMo-2 [47] on MathCAMPS) needs GPU access; DataDecide [44] and the Implicit Curriculum repository [22] provide a no-inference fallback. Since Workbook 1 the planned models have been built rather than planned: a log-space batched BKT with per-item guess and slip, T-SKIRT with Fluid's item parameters converted to the probit scale and then recalibrated per model, and a state-space IRT with drifting per-item residuals (DriftIRT) that replaces the Martin-Quinn-style dynamic IRT of the original plan. The evaluation harness, six baselines, four learner models, the forgetting tests and the per-skill learning curves exist, run on a laptop, and are covered by 429 automated tests (Chapters 6 to 8).

The project aims at a peer-reviewed publication; our advisor identified NeurIPS as the target. The research question is open in the reviewed literature, the baselines are cited NeurIPS and COLM methods, and the success criterion is pre-committed: beat the strongest trivial predictor on the same benchmark subset and horizon. Workbook 1 set that bar at the item-history mean (pooled over six models, log-loss 0.360 and within-skill AUC 0.80 one checkpoint ahead). Chapter 6 reports that three learner models now beat that bar on held-out test checkpoints of OLMo-2 on MMLU by 0.06 to 0.07 log-loss, and also that a one-line recency-weighted item mean recovers most of that gain, so the bar is raised to the recency-weighted item mean for every comparison from Sprint 5 on. Whether the fitted state measures transferable mastery (the advisor's held-out transfer test, Chapter 6.7) and whether mastery-guided training helps are questions still to test. Three risks identified in the review are treated as planned analyses (details in Chapter 4): how separable the 57 MMLU subject trajectories are within a run; whether item-parameter drift is detectable at our data density, to be settled by a simulation-based power analysis rather than assumed; and how fast the trivial baselines decay with forecast horizon. The practical value is direct: a per-skill mastery estimate with uncertainty tells a developer whether a checkpoint has learned a skill or is guessing, whether a later one has forgotten it, and which skills to target with the next training data, which is the use CDS [16] and Skill-it [46] make of skill diagnoses; the visualization requested by our advisor makes that usable without reading the paper. Semester one delivers the pipeline, baselines and first validated learner model on public data; semester two delivers the Phase 2 extension, the drift and forgetting analyses, the visualization, and the paper.

Chapter 3
Identify Baseline Approaches

We group baselines into three tiers: trivial predictors that any learner model must beat on next-checkpoint prediction; published methods that describe the same phenomena without a learner model; and the learner-model family itself, where the question is which member fits our data. For each we identify the state-of-the-art method, the implementation we will use, and its status.

3.1 Tier 1: Trivial next-checkpoint predictors (implemented, results in hand)

These have no latent state. They were implemented in our evaluation harness (src/llm_kt/baselines.py) and scored on all six models of the Phase 1 data using a strict walk-forward protocol: at each origin checkpoint t the predictor sees every response at checkpoints at most t and predicts the probability of a correct answer for every item at checkpoint t+h. Results pooled over six models and all five retained benchmarks, now at four horizons (horizons 10 and 20 added in Sprint 4):

Predictor	**Description**	**h=1 log-loss / AUC**	**h=5**	**h=10**	**h=20**
global_mean	Overall accuracy at the last checkpoint	0.661 / 0.50	0.664 / 0.50	0.667 / 0.50	0.672 / 0.50
skill_mean	Per-skill accuracy at the last checkpoint	0.582 / 0.50	0.585 / 0.50	0.588 / 0.50	0.594 / 0.50
copy_last	Item's last correctness, softened (eps=0.1)	0.432 / 0.73	0.473 / 0.70	0.491 / 0.70	0.539 / 0.67
copy_last_calibrated	Two-state Markov chain on observed correctness, per skill	0.378 / 0.73	0.404 / 0.70	0.423 / 0.70	0.460 / 0.67
item_mean	Item's history average shrunk toward the global mean (inspired by checkpoint averaging [18])	0.360 / 0.80	0.388 / 0.78	0.413 / 0.76	0.463 / 0.73
item_mean_ewma (added Oct 27)	Exponentially weighted item history, weight rho per checkpoint of age (rho = 0.9, half-life 6.6 checkpoints); run so far on OLMo-2 MMLU only	see Chapter 6			

How to read the table. Log-loss and Brier are weighted by item count over (model, target checkpoint, skill) slices, so models with more checkpoints weigh more;

AUC is the unweighted mean over slices where both classes occur. AUC is computed within a skill slice, so any predictor that is constant within a skill (global_mean, skill_mean, and a BKT with one guess/slip pair per skill) scores 0.50 by construction. The pooled numbers cover all five retained benchmarks; on MMLU alone item_mean scores log-loss 0.483 and AUC 0.79 at h=1, and a learner model fitted on MMLU must be compared against that. copy_last_calibrated applies its one-step transition probabilities at every horizon, so its h=5 score is that of a fixed predictor, not a five-step Markov forecast. The pre-committed bar for any learner model in Workbook 1 was: beat item_mean on log-loss and within-skill AUC on the same benchmark subset and horizon. Chapter 6 shows that a recency-weighted item mean (item_mean_ewma, added as a sixth baseline on October 27) beats item_mean by about 0.06 log-loss on the test block of OLMo-2 MMLU and ties every learner model built so far, so from Sprint 5 the bar is item_mean_ewma with its weight chosen on the validation block. DAS3H [7] (open source, github.com/BenoitChoffin/das3h), recommended by the literature review as a learned non-latent baseline, is scheduled for Sprint 6; its time-window features must be redefined over training progress.

3.2 Tier 2: Published descriptive methods (open source, to be reproduced)

- Static IRT per checkpoint: Fluid Benchmarking [8]. Code and data at github.com/allenai/fluid-benchmarking; the release includes fitted 2PL item parameters (a, b) for every item, so a static-IRT ability per checkpoint per skill can be computed directly. This is the psychometric baseline and the item-parameter initialisation for our temporal models. Two uses must be kept separate: as a forecast, the ability is estimated from history up to the origin only and carried forward (static persistence); as a diagnosis, the ability is estimated on part of a checkpoint's items and scored on the held-out items of the same checkpoint. An ability fitted on a checkpoint's answers is never scored on those same answers. Status: the persistence variant is implemented (src/llm_kt/static_irt.py) and scored in Chapter 6; the diagnosis variant is scheduled with the held-out transfer test in Sprint 5.
- Per-skill accuracy curves: MathCAMPS [21]. Code at github.com/gpoesia/mathcamps. Reproduced on our data as raw accuracy per (model, skill, checkpoint) with a 95% Wilson interval (src/llm_kt/curves.py). Status: implemented; figures generated for OLMo-2 across MMLU subjects and for six models on one subject.
- Threshold-based emergence: Implicit Curriculum [22]. Code and per-item results at github.com/KaiserWhoLearns/ElementalTask (MIT). Emergence is the first checkpoint at which accuracy exceeds a threshold. Status: trivial to

compute from our curves table; will be reported alongside model-based emergence.

- Item-level transition counting for forgetting: Harmon et al. [24]. No learner model; forgetting is the count of items correct at t and incorrect at t+h. Status: implemented (src/llm_kt/forgetting.py) as the exact one-sided McNemar test per (model, subject) with Holm and selection adjustment, plus a chance-adjusted check that recovers the model's answer-letter preference from accuracy split by gold letter. Result (Chapter 6.6): none of the seven OLMo-2 candidates survives the adjustment, and the five (model, subject) pairs that do are answer-letter shifts on near-chance models rather than forgetting.
- Static skill profiles: Beyond Scores [13] (multidimensional IRT with a Q-matrix) and cognitive diagnosis [15]. These profile a frozen model; we compare our final-checkpoint profile against a static fit on the same checkpoint to show what the temporal model adds. Status: deferred to Phase 2, where a frozen-model profile on MathCAMPS skills is the natural comparison.

3.3 Tier 3: Learner-model family (the methods under study)

Model	Implementation	Licence / status	Role
BKT with forgetting	Own implementation, src/llm_kt/bkt.py (411 lines, 26 tests): log-space forward-backward with batched per-item emissions, MAP-EM per skill, per-item guess/slip shrunk toward a skill or Fluid anchor with recency weighting	Implemented October 27. pyBKT was not used: its E-step multiplies raw emissions and underflows for batches of hundreds of items; the log-space recursion is tested against brute-force enumeration and a 2,000-item checkpoint.	Interpretable model: per-skill initial mastery, learn, forget, and per-item guess and slip. The one-pair-per-skill limit (kappa = 1e6) is run as a control and scores AUC 0.50 by construction; the per-item version beats the item-mean bar at every horizon (Chapter 6.5).

Model	Implementation	Licence / status	Role
DriftIRT (replaces the Martin-Quinn-style dynamic IRT)	Own implementation, src/llm_kt/drift_irt.py (15 tests): per-skill ability random walk plus per-item residual random walk, diagonal-Gaussian filter, decoupled forecast variance	Implemented October 20. emIRT::dynIRT [30] remains the reference point only: it assigns each item to one period and its variational variances are documented as unusable, so the plan of a tied-parameter Kalman smoother was replaced by this filter, which gives each item the free, drifting number the item mean has while keeping the per-skill state.	Primary temporal-IRT model. Beats the item-mean bar on the test block at horizons 1, 5 and 20; ties the recency-weighted item mean (Chapter 6.5).
T-SKIRT	Own implementation, src/llm_kt/tskirt.py (12 tests) with per-model recalibration (src/llm_kt/recalibrate.py)	Implemented October 5 to 18. Fluid's logistic 2PL parameters are converted to the probit scale (discrimination divided by 1.7); with the published parameters the model loses badly, with per-model recalibrated difficulty and discrimination it ties the bar.	Answers whether temporality helps over static IRT: yes early in training, no at the plateau (Chapter 6.5).
Static IRT persistence	src/llm_kt/static_irt.py (10 tests), with the walk-forward calibration layer src/llm_kt/calibrate.py (12 tests)	Implemented. Per-skill 2PL ability refitted at each origin on a window of history and carried forward; temperature or Platt calibration fitted on past checkpoints only.	Psychometric baseline. Calibration fixes its under-confidence (test log-loss 0.393 to 0.382) but not its ranking (Chapter 6.5).
GD-GPIRT	github.com/yahoochen97/OrdGPIRT [33]	No licence; R/MATLAB; expensive; runnable entry point for the GD variant not yet verified.	Subsample only, for the item-drift question (Sprint 10).
Deep KT (DKT, AKT)	pyKT, github.com/pykt-team/pykt-toolkit	MIT.	Ablation only, on Phase 1 data (six models), with (model, subject) pseudo-learners and a leave-one-model-out split; on Phase 2 only if more than one model is evaluated there.

Model	Implementation	Licence / status	Role
VTIRT	github.com/ yunsungkim0908/vtirt [32]	No licence file; one item per step.	Not adopted; cited.

Four validation steps apply to every Tier 3 model. (1) A parametric-recovery simulation using our own fitted parameters as the generating model over six autocorrelated trajectories, as recommended by Jiang et al. [10], reporting recovery at our actual design. (2) A unit test that the number of state transitions equals the number of checkpoint gaps (T minus 1 for T checkpoints, with an initial state at the first) and that no transition occurs between items at the same checkpoint. (3) Chronological validation and test blocks fixed before model selection, with hyperparameters chosen on the validation block only, so that the exploratory baseline results reported here do not leak into model choice; this is now implemented and every learner-model result in Chapter 6 follows it. (4) Interval coverage checks for any reported uncertainty: Wilson bands describe raw accuracy, not posterior mastery, and clustering by subject does not by itself address dependence across checkpoints or models.

Chapter 4
Dependencies and Deliverables

4.1 Dependencies

Only showstoppers are listed.

- Per-item, per-checkpoint correctness data for Phase 1. Resolved on September 29, 2026: allenai/fluid-benchmarking on HuggingFace contains it for six models (Amber-6.7B, K2-65B, OLMo-1 7B, OLMo-2 7B, Pythia-2.8B, Pythia-6.9B), 452 checkpoints, 27,842 usable items. Ingested into data/processed/fluid_responses.parquet. Fallback if the release were withdrawn: DataDecide-eval-instances (ODC-BY) and the ElementalTask repository (MIT), both verified public.
- GPU access for the preferred Phase 2 (OLMo-2 checkpoints on MathCAMPS). Not yet secured. Roughly 50 to 100 checkpoints of a 7B model on a planned subset of MathCAMPS problems (count to be fixed from the dataset manifest). Options: SJSU HPC cluster, Google Colab Pro, or rented cloud GPUs (Lambda, RunPod). Owner: Aniket. A pilot throughput and cost estimate is due 2026-11-25; the go/no-go decision is due 2026-12-12. If no GPU is available, Phase 2 uses the no-inference fallback datasets above, which lose the Common Core skill taxonomy but keep a skill-decomposed structure; the fallback requires item-ID and coverage validation before it is adopted.
- Skill tags. Phase 1 uses MMLU's 57 subjects, which exist in the item ids. Risk: the MMLU psychometric audit [12] finds STEM and non-STEM abilities correlate at 0.965 across final models (while still being distinguishable), so subject trajectories may move largely together. Mitigation: measure inter-subject trajectory correlation as soon as the first model is fit; fall back to a four-category rollup as the primary unit if needed.
- The dynamic IRT model itself. Resolved on October 20: the tied-parameter Kalman smoother planned in Workbook 1 was replaced by DriftIRT, a state-space IRT whose per-item residuals drift as a random walk, filtered with a diagonal Gaussian approximation (src/llm_kt/drift_irt.py). It is identifiable, stable, runs in about 17 seconds per setting at horizon 1, and beats the bar; the parametric-recovery simulation that validates its state estimates is scheduled for Sprint 5.
- Batched BKT. Resolved on October 27: a log-space forward-backward with batched per-item emissions (src/llm_kt/bkt.py) replaces pyBKT; tested against brute-force enumeration and a 2,000-item checkpoint for underflow.

- Time metadata. Step is missing for Amber and K2, and tokens seen is missing for Amber, K2 and Pythia in the Fluid release. Pythia's can be derived (2,097,152 tokens per step); Amber's and K2's come from LLM360's published schedules. Resolved on October 28 (src/llm_kt/time_metadata.py): Pythia's count is derived exactly from 2,097,152 tokens per step; Amber's and K2's are estimated from their 360 equally spaced checkpoints over 1,259B and 1,300B tokens, and each row records whether its count is published, derived or estimated. Every model so far was scored with the checkpoint index as its clock; T-SKIRT and DriftIRT accept the tokens column as a flag, and the rerun is a Sprint 5 item.
- Benchmark scope (Decision 6, October 27). Only MMLU carries skill labels, so learner models and headline results use MMLU only; ARC, GSM8K, HellaSwag and WinoGrande stay in the table for the baseline ladder, as held-out transfer targets, and as single-skill controls. Phase 2 on MathCAMPS removes this limitation.
- Team availability. Three members with coursework; if one member is unavailable for more than two weeks, the visualization track is the one to defer, since the paper does not depend on it.

4.2 Deliverables

- Canonical dataset and ingest pipeline: a long-format Parquet table of (model, checkpoint, item, skill, correct) built reproducibly from public releases, with a tested ingest script (delivered for Phase 1; Phase 2 extends it).
- Learner-model library: Python package llm_kt with BKT with forgetting, static IRT, T-SKIRT with per-model recalibration, DriftIRT and a walk-forward calibration layer, all exposing one Predictor interface (delivered; deep KT ablation to follow in Sprint 9).
- Evaluation harness: walk-forward next-checkpoint prediction at horizons 1, 5, 10 and 20 with log-loss, Brier, accuracy and AUC per (model, checkpoint, skill), fixed chronological validation and test blocks, subject-clustered bootstrap standard errors and paired differences (delivered); the parametric-recovery simulation follows in Sprint 5.
- Fitted mastery trajectories: a released dataset of per-skill mastery estimates with uncertainty for every checkpoint of every model studied, plus item-parameter drift estimates.
- Interactive visualization: a web application (FastAPI backend serving fitted trajectories, React frontend) showing per-skill mastery over checkpoints with

uncertainty, emergence and forgetting events, item-difficulty drift, and cross-model comparison. Requested by the advisor.

- Analyses: the baseline ladder (delivered: where predictability lives, by model, benchmark, horizon and over training), forgetting events that survive the McNemar test and the chance-adjusted check (delivered: none so far), emergence order versus curriculum order, and item-parameter invariance at bank level with the detectability bound reported.
- Publications: a NeurIPS submission (May 2027) and an arXiv preprint; a public GitHub repository with code, data-building scripts and documentation.

Chapter 5
Project Architecture

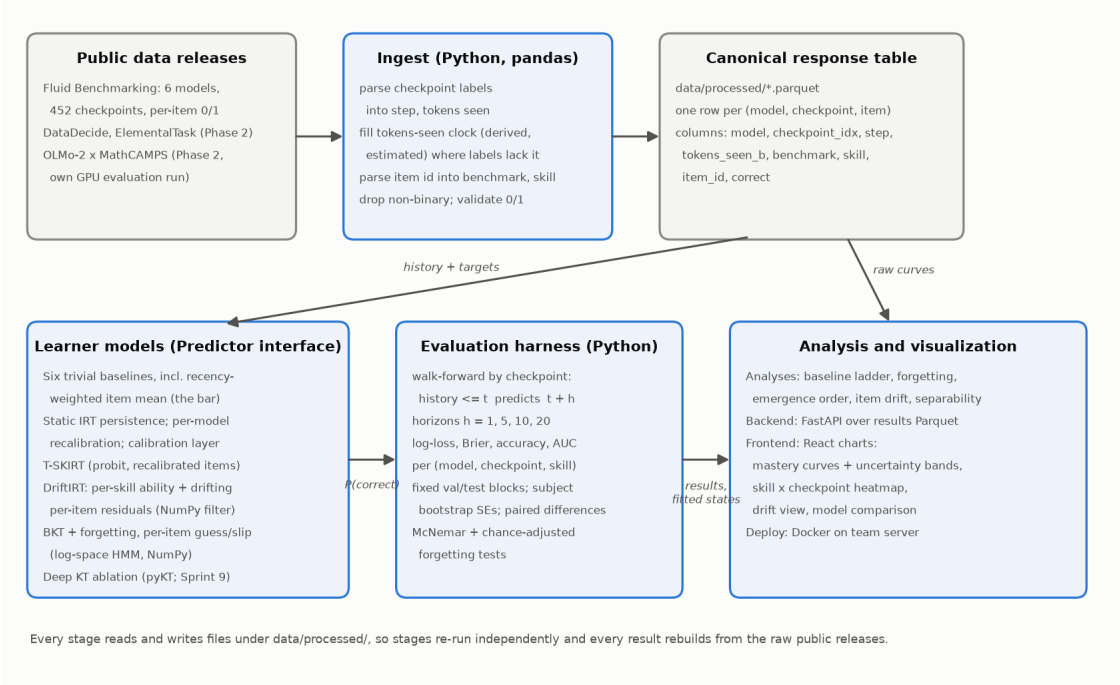


Figure 5.1. System architecture. Left to right: public data releases are ingested into one canonical response table; learner models and baselines read it through a common Predictor interface; the walk-forward harness scores them and writes results; the visualization serves fitted trajectories and results to a browser.

The system is a batch pipeline followed by a serving layer. Each stage reads and writes files in data/processed/, so stages can be re-run independently and results are reproducible from the raw releases.

5.1 Data layer

Function: turn heterogeneous public releases into one table. Sources: Fluid Benchmarking (six per-model CSVs, wide format, checkpoint columns named differently per model family), and in Phase 2 our own OLMo-2 evaluation on MathCAMPS plus optional DataDecide and ElementalTask files. The ingest module (src/llm_kt/ingest.py) parses checkpoint labels into step and tokens-seen, parses item ids into benchmark and skill, drops the non-binary TruthfulQA benchmark, validates that every kept cell is 0 or 1, and writes a long-format Parquet table with columns model, checkpoint_idx, checkpoint_label, step, tokens_seen_b, benchmark, skill, item_id, correct. Technology: Python 3.12, pandas, pyarrow, huggingface_hub. Current size: 12,584,584 rows, 27 MB.

5.2 Model layer

Function: estimate a per-skill knowledge state per checkpoint and predict future responses. Every model implements one interface, Predictor.predict(history, target, horizon) -> probabilities, where history is all responses at checkpoints up to the origin (with step and tokens-seen columns where available), target lists the items to predict, and horizon is the number of checkpoints ahead. Learner models additionally export a fitted-state artifact per (model, skill, checkpoint): filtered and smoothed mastery estimates, their uncertainty, and item-parameter estimates, written to data/processed/ for the analysis and visualization layers. Members, all implemented unless stated: six trivial baselines including the recency-weighted item mean; per-skill accuracy curves with Wilson intervals; static IRT persistence; T-SKIRT with per-model recalibration of item difficulty and discrimination; DriftIRT (per-skill ability and per-item residual random walks); BKT with forgetting and per-item guess and slip (log-space NumPy HMM); a walk-forward calibration wrapper (temperature or Platt) that can sit on top of any predictor; deep KT (planned, pyKT, ablation, Sprint 9). Technology: Python, NumPy, SciPy, pandas; PyTorch for the ablation only.

5.3 Evaluation layer

Function: score every model the same way. The harness (src/llm_kt/evaluate.py) walks forward through each model's checkpoints, calls the predictor with history up to origin t and the horizon h, rejects non-finite or out-of-range probabilities, scores predictions at t+h with log-loss, Brier, accuracy and rank-based AUC per (model, target checkpoint, skill), and writes one results Parquet per horizon. Metrics are implemented without external dependencies (src/llm_kt/metrics.py). Added since Workbook 1: fixed chronological train/validation/test checkpoint blocks per model (20% each for validation and test), subject-clustered bootstrap standard errors in summarize(), paired differences with 95% intervals between any two predictors, horizons 10 and 20, the McNemar peak-to-final forgetting test with Holm and selection adjustment, the chance-adjusted (answer-letter) check, and the baseline-ladder and variance-decomposition analyses (src/llm_kt/ladder.py). Still planned: the parametric-recovery simulation and coverage checks for fitted states (Sprint 5). Technology: Python, NumPy, SciPy, pandas, pytest (429 tests).

5.4 Analysis layer

Function: turn fitted states into the paper's findings. Emergence order per model (model-based and threshold-based), forgetting events that pass the paired test, item-parameter drift at bank level with the detectability bound, inter-subject trajectory

correlation, and cross-model comparison. Technology: Python, matplotlib for static figures.

5.5 Visualization layer

Function: let a developer or researcher browse per-skill mastery over training. Backend: FastAPI serving the fitted-trajectory and results Parquet files as JSON endpoints (per model, per skill, per checkpoint; emergence and forgetting events; item drift). Frontend: React with a charting library, views for mastery-over-checkpoints with uncertainty bands, a skill-by-checkpoint heatmap in the DeepTracker idiom [40], forgetting-event markers, item-difficulty drift, and side-by-side model comparison. Deployment: Docker container on the team's server; static export for the paper's supplementary material. Technology: Python/FastAPI, React, Docker.

5.6 Repository layout

- src/llm_kt/: ingest.py, time_metadata.py, curves.py, metrics.py, evaluate.py, baselines.py, static_irt.py, tskirt.py, recalibrate.py, calibrate.py, drift_irt.py, bkt.py, forgetting.py, ladder.py (about 4,700 lines of library and script code).
- scripts/: ingest_fluid.py, plot_skill_curves.py, run_baselines.py, run_models.py, run_calibrated.py, run_forgetting.py, analyze_sweep.py, analyze_horizons.py, analyze_ladder.py, analyze_drift_irt.py, analyze_bkt.py.
- tests/: 429 pytest tests in 16 files covering parsing, aggregation, metrics, harness protocol, blocks, every baseline and learner model, recalibration, calibration, the forgetting tests and the ladder analysis.
- docs/results/bkt/: stored BKT result tables and parquets so the BKT figures can be regenerated without refitting.
- data/raw/, data/processed/: gitignored; rebuilt by scripts.
- docs/: project notes (Decisions 1 to 6, BKT write-up), abstract v2, research brief, literature review, team and advisor briefings, figures, workbooks and their reviews, presentation script, individual log books.
- meetings/: one note per advisor or team meeting.

Chapter 6
Evaluation Methodology

This chapter is the evaluation protocol as implemented and the results it has produced so far. Everything below runs from the repository with the commands listed in Chapter 7; every number is read from a stored result file (data/processed/ or docs/results/) and was re-verified from those files on October 28, 2026 before being written here. Unless stated otherwise, learner-model results are for OLMo-2 7B on MMLU (Decision 6: MMLU is the one benchmark with skill labels), and the baseline table in Chapter 3 is the one place where all six models and all five benchmarks are pooled.

6.1 Protocol: walk-forward next-checkpoint prediction

The unit of time is the training checkpoint (Decision 1). For a model with T checkpoints and a horizon h, the harness (src/llm_kt/evaluate.py) forms every (origin, target) pair with target = origin + h and at least two checkpoints of history before the origin. For each pair the predictor receives the full response history up to and including the origin and a list of target items, and returns one probability of a correct answer per item. Predictions are then scored against the observed responses at the target checkpoint. Nothing after the origin is visible to the predictor, and the harness rejects probabilities that are non-finite or outside (0, 1). The same loop scores every predictor, so no predictor can be advantaged by how it is scored.

Fixed chronological blocks. Before any learner model was selected, each model's checkpoints were split chronologically into a training block (first 60%), a validation block (next 20%) and a test block (last 20%); for OLMo-2's 94 checkpoints that is checkpoints 0 to 55, 56 to 74 and 75 to 93. Hyperparameters are chosen on the validation block only; test-block numbers are reported, never ranked, and were looked at only after the choice was made. 'Pooled' numbers average over every target checkpoint in the run and are shown alongside, because early checkpoints behave differently from late ones (Section 6.4).

Horizons. Every predictor is scored at h = 1, 5, 10 and 20 checkpoints ahead. Horizon 1 is inflated because adjacent checkpoints are near-identical; horizon 20 on OLMo-2 spans a quarter of the run. Results are reported per horizon and never averaged across horizons.

6.2 Metrics and uncertainty

- Log-loss (primary): the mean negative log-likelihood of the observed response under the predicted probability; lower is better. It rewards both ranking and calibration.
- Within-skill AUC: the rank AUC computed inside each (model, target checkpoint, skill) slice and averaged over slices where both classes occur; 0.5 is a coin flip. Because it is computed within a skill, any predictor that is constant within a skill scores exactly 0.5, which is the test that a per-skill model must pass.
- Brier score and accuracy are stored for every slice and reported in the result tables; they never change a conclusion that log-loss and AUC reach, so the text quotes the two primary metrics.
- Standard errors: a bootstrap over the 57 MMLU subjects (200 resamples, subjects resampled with replacement, the same resample applied to every predictor). Subjects are the sampling unit because responses within a subject are correlated across items and checkpoints. The '±' in every table is this standard error.
- Paired differences: for two predictors scored on identical rows, the difference in log-loss (or AUC) with a 95% bootstrap interval over subjects. If the interval contains zero the two predictors tie. All 'beats' and 'ties' in this chapter are statements about these intervals.
- Selection rule: the setting with the lowest validation log-loss is chosen; the test block is scored once for that setting. Where a grid optimum sat at a grid edge, the grid was extended once (BKT kappa) or the edge is reported as a caveat (DriftIRT q_theta).

6.3 Baselines and the pre-committed bar

Six baselines with no latent state set the bar (Chapter 3, Tier 1). Workbook 1 fixed the comparison rule in advance: a learner model must beat the item-history mean (item_mean) on log-loss and within-skill AUC on the same benchmark subset and horizon. On the OLMo-2 MMLU slice the bar is log-loss 0.457 and AUC 0.862 pooled over the run, and 0.375 ± 0.015 and 0.916 ± 0.006 on the test block, one checkpoint ahead. A sixth baseline, the exponentially weighted item mean (item_mean_ewma, weight rho = 0.9 per checkpoint of age, chosen on validation), was added on October 27 after the first learner-model results; its role is explained in Section 6.5.

6.4 The baseline ladder: where predictability lives

The advisor asked on October 19 for the simplest result to be examined from every angle: on OLMo-2 MMLU one checkpoint ahead, guessing from the model's overall score so far gives log-loss 0.69, guessing from its score on the subject gives 0.63, and guessing from the specific question's own history gives 0.45. Knowing the subject helps a little; knowing the question helps a lot. The ladder analysis (src/llm_kt/ladder.py, scripts/analyze_ladder.py) quantifies this with two gaps: the skill gap (global mean minus skill mean) and the item gap (skill mean minus item mean), both in log-loss.

Horizon	Global mean	Skill mean	Item mean	Skill gap	Item gap
1	0.695	0.632	0.454	0.063	0.178
5	0.694	0.638	0.479	0.056	0.160
10	0.695	0.644	0.499	0.050	0.145
20	0.692	0.649	0.536	0.044	0.113

Table 6.1. The ladder on OLMo-2 MMLU by horizon (pooled log-loss). Both gaps shrink with horizon only because the item mean degrades while the global mean is flat: item-level knowledge is the perishable part. Across all six models and benchmarks the item gap goes from 0.222 at h = 1 to 0.131 at h = 20 while the skill gap stays at 0.079.

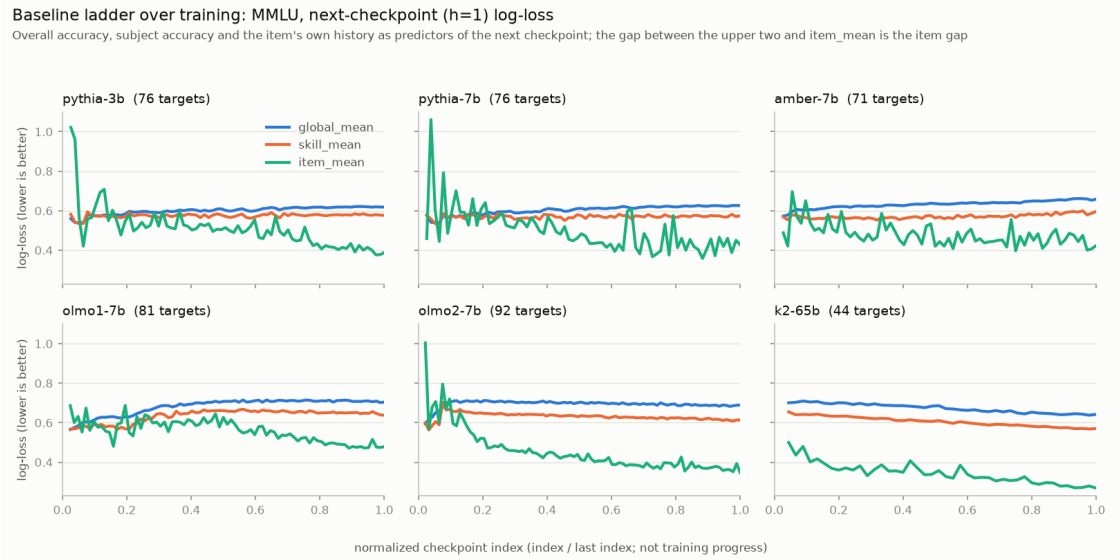


Figure 6.1. The ladder over training on MMLU, one checkpoint ahead, one panel per model. The item gap grows as training proceeds: early in training the item's history is useless (worse than the subject mean at eight early OLMo-2 checkpoints), and it becomes the dominant predictor once per-item behaviour stabilises. OLMo-2's item gap averages 0.074 over the first third of its checkpoints and 0.242 over the last third (Spearman 0.92 with checkpoint index); every model shows the same direction (Spearman 0.68 to 0.93).

- Per model (MMLU, h = 1): the item gap exceeds the skill gap for every model, by a factor of 1.5 (Amber, OLMo-1) to 3.9 (K2). Amber and the two Pythia models have a skill gap of 0.03 or less: they have no subject knowledge on MMLU to exploit.
- Per benchmark (OLMo-2, h = 1): the item gap is largest on ARC-Challenge (0.40), HellaSwag (0.36) and WinoGrande (0.27), then MMLU (0.18). GSM8K reverses the order (skill gap 0.50), but outside MMLU 'skill' is the whole benchmark, so that gap only says GSM8K accuracy is far below the pooled accuracy. Only MMLU tests subject against item, which is one reason for Decision 6.
- Variance decomposition (MMLU): at the final OLMo-2 checkpoint the subject explains 12.7% of the variance in item correctness (law of total variance over subjects); the item's own running mean up to the previous checkpoint explains 58.3% (R-squared over items). Medians over the run are 10.9% and 49.5%. For Pythia and Amber the subject share is under 1.1%.
- Caveat: part of the item gap is 'whole history versus last checkpoint', because global_mean and skill_mean use the last checkpoint while item_mean uses the full history. A subject-level running mean would separate the two; it is scheduled with the Sprint 5 rescoring.

6.5 Learner-model results

Four learner models have been built and scored under the protocol above (Chapter 7 gives their definitions). They are presented in the order they were built, because each one's result shaped the next.

Static IRT and T-SKIRT with borrowed item parameters lose

Fluid Benchmarking ships a difficulty and a discrimination for every item, fitted on 102 finished models. A per-skill ability estimated from a window of history and carried forward (static IRT persistence), and T-SKIRT, which lets that ability drift and discounts old evidence, were scored with those parameters. T-SKIRT at its best drift setting scores pooled log-loss 0.561 and AUC 0.804 against the bar's 0.457 and 0.862. The diagnosis, confirmed by the next experiment, is that parameters fitted across finished models do not describe one model's trajectory: an item that is 'easy' on average can be hard for OLMo-2 at checkpoint 40.

Per-model recalibration closes most of the gap

Recalibration (src/llm_kt/recalibrate.py) re-learns each item's difficulty offset and log-discrimination multiplier from the model's own past checkpoints by Newton maximum-a-posteriori steps with shrinkage toward the published values, using only history up to the origin. With recalibration T-SKIRT is a different model:

Predictor (h = 1)	Pooled log-loss	Pooled AUC	Test log-loss	Test AUC
Item mean (the bar)	0.457	0.862	0.375 ± 0.015	0.916 ± 0.006
T-SKIRT, recalibrated (settings chosen on validation)	0.446	0.858	0.375 ± 0.024	0.912 ± 0.006
Static IRT persistence, recalibrated			0.393	0.901
Static IRT persistence, recalibrated + calibration layer			0.382	0.901

Table 6.2. Paired difference on the test block, T-SKIRT minus item mean: log-loss −0.0003 with 95% interval [−0.020, +0.025], a tie; AUC −0.003 [−0.006, −0.001], the item mean slightly but reliably better at ranking. Pooled over the run T-SKIRT leads by 0.01 to 0.03 log-loss, growing with horizon (Table 6.3). A sweep of the three dials (shrinkage of difficulty and of discrimination, drift rate), selected on validation only, plateaus at validation log-loss 0.377 against the item mean's 0.386; the walk-forward calibration layer (temperature or Platt scaling fitted on past checkpoints) fixes static IRT's under-confidence and changes nothing for T-SKIRT (0.375 to 0.375). The remaining gap was ranking within a skill, not confidence, and its cause was structural:

the item mean has one free number per item that updates every checkpoint, T-SKIRT has one ability per skill plus a fixed per-item offset.

Horizon	Item mean, test [pooled]	T-SKIRT, test [pooled]	Paired difference on test (T-SKIRT minus item mean)
1	0.375 [0.457]	0.375 [0.446]	−0.000 [−0.020, +0.025], tie
5	0.382 [0.484]	0.382 [0.463]	−0.001 [−0.014, +0.012], tie
10	0.392 [0.506]	0.399 [0.483]	+0.007 [−0.003, +0.015], tie
20	0.414 [0.545]	0.437 [0.518]	+0.024 [+0.016, +0.031], item mean wins

Table 6.3. Recalibrated T-SKIRT further ahead (log-loss). The temporal model wins during the early, fast-changing part of training (pooled) and ties or loses on the late test checkpoints, where learning has plateaued and its forecast variance only spreads predictions toward 0.5.

DriftIRT: give the model the free per-item number

DriftIRT (src/llm_kt/drift_irt.py, October 20) keeps the per-skill ability random walk and adds a per-item residual that also drifts as a random walk, shrunk toward the item's published difficulty when first seen, with the forecast variance decoupled from the fitting drift. A grid of 48 settings was scored on the validation block; the best (q_theta 0.01, q_delta 0.01, residual prior variance 1, plug-in forecast, residual on the probit scale) has validation log-loss 0.321 against the item mean's 0.386.

Horizon	DriftIRT, test (± SE)	Item mean, test (± SE)	Paired log-loss difference	AUC: DriftIRT vs item mean
1	0.305 ± 0.020	0.375 ± 0.015	−0.069 [−0.080, −0.056]	0.933 vs 0.916
5	0.316 ± 0.020	0.382 ± 0.014	−0.067 [−0.077, −0.052]	0.929 vs 0.912
20	0.347 ± 0.019	0.414 ± 0.013	−0.066 [−0.078, −0.052]	0.915 vs 0.898

Table 6.4. DriftIRT beats the bar on the test block at every horizon, with intervals well clear of zero, and unlike T-SKIRT it does not degrade at horizon 20 (pooled: 0.385 vs 0.457 at h = 1). Two audits were run before this was accepted: fresh fits from raw data at origins 60, 85 and 92 reproduce the harness rows to four decimals (no cache leakage), and shuffling the target labels within each skill raises log-loss from 0.31 to about 1.4 (the predictions are not reading the target).

BKT with per-item guess and slip

BKT (src/llm_kt/bkt.py, October 27) is the classic two-state model of Decision 2 adapted to batched checkpoints: a log-space forward-backward over the checkpoints of each subject, per-item guess and slip shrunk toward a skill-shared or Fluid-derived anchor by a pseudo-count kappa, with recency weight rho per checkpoint of age. A 61-setting grid was scored on the validation block; the chosen setting is the skill anchor with kappa 0.1 and rho 0.9. The grid also contains the one-pair-per-skill limit (kappa = 1e6), which is classic BKT.

Predictor (h = 1)	**Pooled log-loss**	**Pooled AUC**	**Test log-loss**	**Test AUC**
Item mean (the bar)	0.457	0.862	0.375 ± 0.015	0.916 ± 0.006
Recency-weighted item mean, rho 0.9 (control)	0.403	0.880	0.317 ± 0.019	0.933 ± 0.006
BKT, per-item, skill anchor, kappa 0.1, rho 0.9	0.411	0.880	0.310 ± 0.023	0.933 ± 0.006
BKT, per-item, Fluid anchor, kappa 1, rho 0.9	0.388	0.882	0.316 ± 0.022	0.929 ± 0.007
BKT, classic (one guess/slip pair per skill)	0.637	0.500	0.623 ± 0.011	0.500

Table 6.5. Paired differences on the test block: BKT (skill anchor) minus item mean, log-loss −0.065 [−0.078, −0.045] and AUC +0.017 [+0.014, +0.020], BKT wins on both; the classic limit minus item mean, +0.248, and its AUC is 0.50 by construction, exactly as Workbook 1 predicted. BKT keeps its lead over the item mean at horizons 5 and 20 (test log-loss 0.323 vs 0.382 and 0.359 vs 0.414). These numbers were reproduced on a second machine on October 28 (two settings, about 14 minutes of CPU): log-loss agreed with the stored tables to six decimal places.

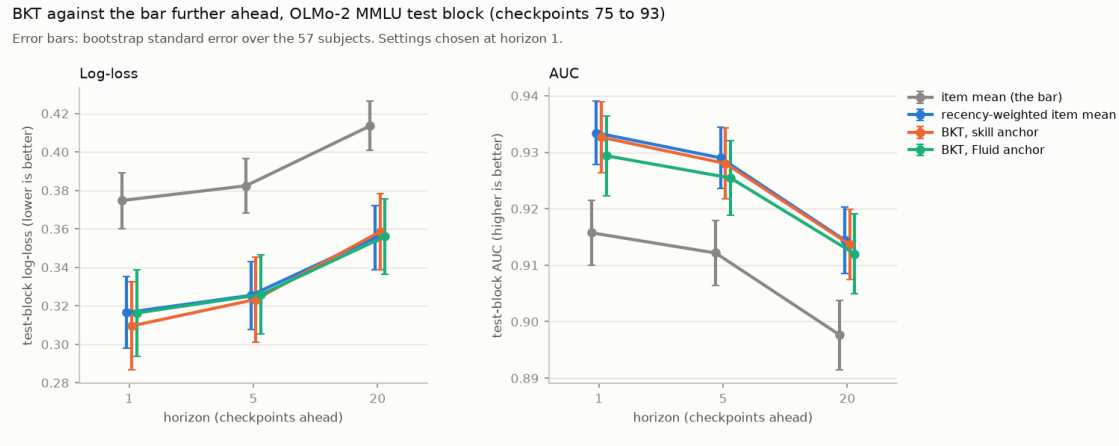


Figure 6.2. Test-block log-loss by horizon for the item mean, the recency-weighted item mean and the two chosen BKT settings, with bootstrap standard errors over subjects. BKT beats the item mean at every horizon and is indistinguishable from the recency-weighted item mean.

The control that reframes the result

Both DriftIRT and BKT were checked against a control the plan did not contain: an exponentially weighted item mean with the same recency weight (rho 0.9, half-life about 6.6 checkpoints), chosen on validation. It scores 0.317 on the test block, so it recovers about 87% of either learner model's gain over the plain item mean. Against it, BKT ties at every horizon (−0.007 [−0.013, +0.002] at h = 1; +0.003 [−0.002, +0.011] at h = 20), and DriftIRT holds a small, real lead of −0.009 [−0.012, −0.006] log-loss with equal AUC. The Fluid-anchored BKT leads the control pooled over the run (by 0.015 to 0.051 log-loss, intervals excluding zero), because a published-parameter prior is worth something while an item has only a handful of observations, and ties it on the late test block.

Model	**vs item mean (test, h = 1)**	**vs recency-weighted item mean (test, h = 1)**	**Horizon 20 (test)**
T-SKIRT, recalibrated	tie (−0.000)	not yet scored	loses to item mean (+0.024)
DriftIRT	wins (−0.069)	wins, small (−0.009)	wins (−0.066 vs item mean)
BKT, per-item	wins (−0.065)	tie (−0.007)	wins (−0.055 vs item mean); tie vs control
BKT, classic	loses (+0.248)	loses	loses

Table 6.6. Summary across learner models (paired log-loss differences, negative favours the learner model). The three per-item models agree on one finding: recent per-item state is what matters; per-skill state adds a small, reliable amount on top of it

(DriftIRT) or nothing measurable (BKT). From Sprint 5 the pre-committed bar is therefore the recency-weighted item mean, and T-SKIRT and DriftIRT are rescored against it on validation before any claim is made.

6.6 Forgetting: the McNemar test and the chance-adjusted check

Workbook 1 listed seven OLMo-2 subjects whose accuracy fell by more than 0.08 from peak to final checkpoint and promised a paired test with selection control. The test (src/llm_kt/forgetting.py, scripts/run_forgetting.py) is the exact one-sided McNemar test on the items that flip right-to-wrong versus wrong-to-right between a subject's best checkpoint (searched from checkpoint 5 onward) and its last, run for every (model, subject) pair, with Holm adjustment across the 57 subjects and an additional adjustment for having chosen the peak after looking. The seven candidates have raw p-values between 0.002 and 0.049 and adjusted p-values of 1.0: they do not survive. Five (model, subject) pairs across all six models do survive at adjusted $p < 0.05$, among them OLMo-1 on moral_scenarios (895 items, accuracy 0.37 to 0.24 between checkpoints 79 and 82, adjusted $p < 0.0001$).

The surviving pairs were then given a chance-adjusted check, because four of them are on models near chance on MMLU. Fluid stores only correctness, so the model's answer-letter preference was recovered indirectly: for a guessing model, accuracy on items whose gold answer is 'D' equals how often it picks 'D'. Gold letters were matched for all 14,042 MMLU items from the public test split. The check reports how much of the variance in correctness the gold letter explains at the peak and the final checkpoint, a G-test for a change in letter preference, and the drop in accuracy expected from the letter shift alone. Result: at their peak, Pythia-2.8B and Pythia-6.9B on security_studies, Pythia-6.9B and Amber on professional_medicine answered 'D' to essentially every item (98 to 100% accuracy on gold-D items, 0 to 2% on the rest); both subjects have 40 to 45% of gold answers on 'D', so the 'peak' is a letter preference that later moved to 'A' or 'B'. The letter shift accounts for 0.17 to 0.26 of each 0.17 to 0.24 drop, leaving residual drops of at most 0.015. OLMo-1 on moral_scenarios is a one-checkpoint spike over a model that answers 'A' to every item (accuracy 0.238 on both sides of the spike, 100% on gold-A items at the final checkpoint); there is no stable learned state to forget. Conclusion for the paper: no forgetting claim survives on Phase 1 data, and any future claim on a near-chance model must pass the letter check first. A correctness-only null (the drop's percentile among the other 56 subjects) would have pointed the wrong way, because letter shifts cancel when pooled over MMLU but hit individual subjects according to their gold-letter mix.

6.7 What the evaluation has established, and what it has not

- Established: the protocol works end to end on 12.6 million responses, every model is scored identically, and all headline numbers come with subject-level intervals and paired comparisons on checkpoints never used for tuning.
- Established: borrowed item parameters fail for a single training run, per-model recalibration fixes that, and per-item state with recency is the dominant source of predictability. Three learner models beat the Workbook 1 bar on held-out checkpoints at every horizon.
- Established: none of the forgetting candidates is real; four are answer-letter artefacts of near-chance models. This is a negative result worth reporting, and it motivates a chance-aware forgetting definition.
- Not yet established: whether the per-skill state carries information beyond recent per-item history. DriftIRT's lead over the recency control (0.009 log-loss) is real but small; the planned tests are (a) rescoring every model against the recency bar on validation, (b) fitting on the other five models (so far only OLMo-2), (c) the advisor's held-out transfer test: fit the per-skill trajectory on one item set and predict the same checkpoints' performance on held-out items and on other benchmarks, which only a skill-level state can do, and (d) pooling learn and forget rates across models in BKT.
- Not yet established: calibration of the fitted mastery states themselves (coverage of their uncertainty), to be checked by the parametric-recovery simulation in Sprint 5; and anything on MathCAMPS skills, which is Phase 2.
- Threats to validity recorded: MMLU subjects are topics rather than skills and share most of their variance; only one model has been used for model selection; horizon sets are not matched in size (fewer target checkpoints at h = 20); early-checkpoint statistics on Pythia and Amber are dominated by letter bias.

Chapter 7
System Design/Methodology

7.1 Solution overview

The system treats a language model's training run as a student's learning history and asks what the student knows, skill by skill, at each saved checkpoint, validating the estimate by forecasting the student's answers at later checkpoints. Six design decisions, recorded with options and trade-offs in docs/project-notes.md and adopted at team meetings, fix the method:

- Decision 1 (Sep 29): the time step is the checkpoint, not the item. The model does not learn from the benchmark; all items at a checkpoint are simultaneous noisy readings of the skill state, so each learner model makes one state transition per checkpoint gap and absorbs the whole batch of responses at once.
- Decision 2 (Sep 29): primary models are simple and interpretable (BKT with forgetting, temporal IRT); neural knowledge tracing is an ablation only, because there are six learners, not thousands.
- Decision 3 (Sep 29): Phase 1 data is the Fluid Benchmarking release; Decision 4: Phase 1 skills are the 57 MMLU subjects (TruthfulQA dropped); Decision 5: (model, subject) pseudo-learners are used only for the deep-KT ablation with a leave-one-model-out split.
- Decision 6 (Oct 27, from the advisor's suggestion): learner models and headline results use MMLU only; ARC, GSM8K, HellaSwag and WinoGrande stay in the data table for the baseline ladder, as held-out transfer targets and as single-skill controls, and are filtered at the experiment level, never in ingest.

Methodologically the project is an empirical comparison under a pre-registered rule: every model is scored by the same walk-forward harness, selection happens on validation checkpoints, test checkpoints are touched once, and a learner model is kept only if it beats the strongest no-learning predictor with a subject-level interval that excludes zero. The first semester's job is to find which learner model earns its place on public data; the second semester applies the winner to skill-tagged data and builds the visualization.

7.2 Data representation

One long-format table, data/processed/fluid_responses.parquet, holds every response: model, checkpoint index, checkpoint label, training step and tokens seen where published, benchmark, skill, item id, and a 0/1 correctness. It is built from the six per-model Fluid CSVs by src/llm_kt/ingest.py, which parses the checkpoint labels

(step and token counts differ in format per model family), parses the MMLU item ids into subjects, drops TruthfulQA (continuous scores), and asserts that every kept cell is 0 or 1. Item parameters (2PL difficulty b and discrimination a per item, fitted by AI2 on 102 finished models) are loaded by src/llm_kt/static_irt.py and converted to the probit scale (a divided by 1.7) where a model uses a probit link. Every experiment filters this table by model and benchmark through command-line flags; the table itself is never edited.

7.3 Predictors

Every predictor implements one interface, Predictor.predict(history, target, horizon), returning a probability per target row. The notation below: y_it is the response of item i at checkpoint t; s(i) is the item's skill; a_i and b_i are the published discrimination and difficulty; Phi is the standard normal distribution function and sigma the logistic function.

- Item mean: the mean of y_i over all checkpoints up to the origin, shrunk toward the global mean with one pseudo-observation. Recency-weighted item mean: the same with weight rho^(origin − t) on each past checkpoint; rho = 0.9 was chosen on validation. These are the bar.
- Static IRT persistence: at each origin, for each skill, a maximum-a-posteriori ability theta_s is fitted to the responses of the last window of checkpoints under P(y_it = 1) = sigma(a_i (theta_s − b_i)), with a N(0, 1) prior, and carried forward unchanged to the target.
- T-SKIRT [31]: ability follows a Wiener process in time, so a response observed d checkpoints before the origin has effective discrimination a_i / sqrt(1 + a_i^2 nu^2 d), where nu is the drift rate; theta_s is fitted by Newton steps under the probit link P(y = 1) = Phi(a_eff (theta_s − b_i)), and the forecast at horizon h inflates the posterior variance by h nu^2 before folding it into the probit.
- Per-model recalibration (used by static IRT and T-SKIRT): each item's difficulty becomes b_i + delta_i and its discrimination a_i exp(gamma_i), with (delta, gamma, theta) fitted jointly by alternating Newton steps on the history, under Gaussian priors on delta and gamma whose variances tau and tau_a are the shrinkage dials chosen on validation; gamma is clipped to [−2, 2]. Refitted at every origin from history only.
- DriftIRT: state per skill theta_s(t) and per item delta_i(t), each a Gaussian random walk with per-step variances q_theta and q_delta; delta_i starts at N(0, P) when the item is first seen. The filter does, per checkpoint, a predict step (add q to each variance) and an update step (two or three rounds of coordinate-wise Newton updates of theta_s given the deltas and of all deltas given theta_s,

vectorised over items), keeping a diagonal Gaussian posterior. The observation model is P(y = 1) = Phi(a_i (theta_s − b_i) − delta_i) (residual on the probit scale; the difficulty-scale variant Phi(a_i (theta_s − b_i − delta_i)) is a flag and was worse, because the median probit discrimination on MMLU is 0.28). The forecast uses the last posterior means with variance f (V + h q), where f is a separate dial; f = 0 was selected. The filter caches its state across origins and refilters from scratch whenever the history does not extend the cached one.

- BKT with per-item emissions: per skill, a two-state hidden Markov model with P(z_0 = 1) = pi, learn and forget transition probabilities applied once per checkpoint gap, and emissions P(y_it = 1 | z_t = 1) = 1 − slip_i and P(y_it = 1 | z_t = 0) = guess_i, with the log-emission of a checkpoint equal to the sum over its observed items. Fitting is MAP-EM per skill at every origin, warm-started: the skill rates get a weak pseudo-count prior; each item's guess and slip are posterior-mean updates from its (recency-weighted, rho) counts shrunk toward an anchor with pseudo-count kappa. The anchor is either the skill's own EM-estimated shared guess/slip (anchor = skill) or the values implied by the 2PL parameters at two global ability levels (anchor = fluid: guess = sigma(a_i (theta_lo − b_i)), 1 − slip = sigma(a_i (theta_hi − b_i))). The forecast pushes the filtered state h steps through the transition and predicts P(z = 1)(1 − slip_i) + P(z = 0) guess_i. kappa at or above 1e5 pins every item to its anchor and reproduces classic per-skill BKT.
- Calibration layer: any predictor can be wrapped so that its probabilities are mapped through a temperature or a Platt (logistic) transform fitted on the predictor's own past predictions and outcomes at the k most recent origins, so the mapping never sees the target.

7.4 Evaluation and analysis components

- Harness (src/llm_kt/evaluate.py): rolling origins, block assignment, per-slice scoring, summarize() with subject bootstrap, paired_difference(). Results are one Parquet per run with one row per (predictor, model, origin, target, block, skill).
- Forgetting (src/llm_kt/forgetting.py): peak-to-final McNemar table with Holm and selection adjustment; chance_check() (accuracy relative to chance with Wilson intervals, the drop's percentile among other subjects, rest-of-MMLU flip rates at the same checkpoints); letter_check() and match_mmlu_gold() (gold letters from the public MMLU test split matched by subject and prompt text; per-letter accuracy; G-test; letter-expected accuracy and residual drop).

- Ladder (src/llm_kt/ladder.py): pooled ladder table by model, benchmark and horizon; per-checkpoint ladder; between-skill versus within-skill variance decomposition and the item-history R-squared, each tested against hand-computed values on synthetic frames.
- Analysis scripts (scripts/analyze_*.py): select on validation, compare with the bar and the control, write tables and figures; analyze_bkt.py stores every table under docs/results/bkt/ so figures can be redrawn without refitting.

7.5 Implementation steps (reproducible)

The following sequence rebuilds every result in Chapter 6 from the public release on a laptop with no GPU. Commands are run from the repository root inside the project's Python 3.12 virtual environment (pip install -e '.[dev]'); each writes to data/processed/ and can be re-run alone. Times are wall-clock on a 16-core laptop.

- Step 1, data (about 1 minute): python scripts/ingest_fluid.py downloads the six Fluid CSVs and writes fluid_responses.parquet (12,584,584 rows). python scripts/plot_skill_curves.py draws the per-skill curves.
- Step 2, baselines (about 8 minutes per horizon): python scripts/run_baselines.py --horizon H for H in 1, 5, 10, 20 scores the five original baselines on all six models and benchmarks. python scripts/analyze_ladder.py then writes the ladder tables, the variance decomposition and Figure 6.1.
- Step 3, IRT models on OLMo-2 MMLU (seconds to minutes per setting): python scripts/run_models.py --models olmo2-7b --benchmarks mmlu --predictors item_mean static_irt tskirt --nu 0.05 0.2 0.5 --horizon 1; add --recalibrate --tau 1 --tau-a 0.5 for the recalibrated variants and --horizon 5, 10, 20 for the horizon study; python scripts/analyze_sweep.py and python scripts/analyze_horizons.py summarise them. python scripts/run_calibrated.py applies the calibration layer.
- Step 4, DriftIRT (about 17 seconds per setting): python scripts/run_models.py --models olmo2-7b --benchmarks mmlu --predictors drift_irt item_mean --q-theta 0.01 0.05 --q-delta 0.001 0.01 0.05 --delta-prior-var 0.5 1 --forecast-var-scale 0 1 --residual-scale difficulty probit --horizon 1 --tag drift_grid; python scripts/analyze_drift_irt.py selects on validation and reports the test block and horizons 5 and 20.
- Step 5, BKT (about 10 minutes per setting at horizon 1, 70 minutes at horizon 20): python scripts/run_models.py --models olmo2-7b --benchmarks mmlu --predictors bkt item_mean skill_mean --kappa 0.03 0.1 0.3 1 5 20 100 1e6 --rho 1.0 0.9 0.7 --anchor skill fluid --theta-lo -1 -2 --theta-hi 1 2 --horizon 1 --tag bkt_grid (split over several --tag values to run in parallel); python scripts/

run_models.py --predictors item_mean_ewma --rho 0.9 for the control; python scripts/analyze_bkt.py --mastery writes docs/results/bkt/ and the four BKT figures. The full grid took one evening; the two chosen settings reproduce in about 14 minutes.

- Step 6, forgetting (about 1 minute): python scripts/run_forgetting.py --min-peak-idx 5 --chance-check writes forgetting_peak_to_final.parquet and forgetting_chance_check.parquet (the first use downloads the 3.5 MB MMLU test split for gold letters; --skip-letter runs without it).
- Step 7, checks: pytest (429 tests, about 6 seconds) and ruff check . must pass before any change is shared. Each pull request lists the commands that regenerate its numbers.

Design properties that made the results trustworthy: one interface for every predictor, so a bug in scoring would affect all of them equally; probability checks in the harness; blocks assigned before model selection; per-model recalibration and calibration fitted only on history; fresh-fit and shuffled-label audits for any model that beats the bar; and stored result files that every number in this workbook was read from.

Chapter 8
Implementation Plan and Progress

Changes relative to the Workbook 1 submission of October 1, 2026 are shown in bold in this chapter and the next.

8.1 Programming and execution environment (done)

- Python 3.12 virtual environment at the repository root; package llm-kt installed in editable mode from pyproject.toml with dependencies huggingface_hub, pandas, pyarrow, numpy, matplotlib **and scipy (added for the probit link, logsumexp and the Newton solvers)**; dev dependencies pytest and ruff.
- Repository layout established (src/, scripts/, tests/, data/, docs/) with data directories gitignored so raw and derived data are always rebuilt from public sources. **Stored BKT result tables under docs/results/bkt/ are the one exception, kept so figures can be redrawn without a 3-hour refit.**
- Continuous checks: pytest (**429 tests passing, up from 52**) and ruff (clean) run locally before any change is shared. **Work flows through branches and pull requests (ronak_branch, bhagyesh_branch; PRs #1 to #5 merged into main by October 27).** GitHub Actions CI to run both on every push is still planned (Sprint 5).

8.2 Development tools (per member)

Status as of **October 28, 2026**. "Verified" means the tool has been installed and used on that member's machine to produce an artifact present in the repository; "to set up" means it is planned with a date and is not yet acquired.

- Ronak (data/infrastructure, harness, visualization). Verified: Git and GitHub, VS Code, Python 3.12 virtual environment with the project package, pytest, ruff, HuggingFace hub client, pandas/pyarrow, **SciPy**, matplotlib, Node.js 22, WeasyPrint (all used to build the ingest, harness, **learner models, analyses**, figures and **both workbooks**). To set up: FastAPI and React scaffolding (Sprint 7), Docker deployment on the team server (Sprint 12).
- Bhagyesh (learner models). Verified 2026-09-30: Git and GitHub (repository owner), Python 3.12 virtual environment with the project package, pytest and ruff on a fresh clone; Node.js (docx and pptxgenjs) used to build the abstract document and presentation. **Verified 2026-10-27: NumPy/SciPy for the log-space BKT, matplotlib for the BKT figures, and the full 61-setting grid run on his laptop (about 10 minutes per setting at horizon 1).** To set up: pyKT

and PyTorch for the ablation in Sprint 9. **R with emIRT is no longer needed: the dynamic IRT reference fit was dropped when DriftIRT replaced the tied-parameter Kalman smoother.**

- Aniket (data analyses, statistical tests, Phase 2 evaluation). **Verified 2026-10-28: Git and GitHub, Python 3.12 virtual environment with the project package, NumPy/SciPy, pytest and ruff (used to build the forgetting tests, the static IRT baseline and the time-metadata fill); HuggingFace datasets client for the MMLU gold-letter match.** To set up: lm-evaluation-harness and vLLM with a GPU environment for the Phase 2 OLMo-2 runs once access is secured (**pilot by 2026-11-25;** decision by 2026-12-12).

8.3 Hardware, prototyping and simulation environment

- Phase 1 needs no GPU. All ingest, baselines, **learner models, forgetting tests and analyses** run on a laptop: the full six-model ingest takes 47 seconds; the five-baseline evaluation over 452 checkpoints takes about eight minutes per horizon; **T-SKIRT and DriftIRT take seconds to about 17 seconds per setting on OLMo-2 MMLU; BKT takes about 10 minutes per setting at horizon 1 and 70 minutes at horizon 20; the 61-setting BKT grid took one evening in eight parallel processes; the BKT headline run was reproduced on a second machine in 14 minutes.** Figures take seconds.
- Phase 2 (OLMo-2 on MathCAMPS) needs GPU time for roughly 50 to 100 checkpoints of a 7B model. Options under evaluation: SJSU HPC, Colab Pro, rented cloud GPUs. A quote will be obtained before the semester 1 report **(pilot and cost estimate due 2026-11-25)**. The no-GPU fallback datasets (DataDecide, ElementalTask) are verified public.
- The team's self-hosted server (Docker, GitHub Actions to ghcr.io) will host the visualization.
- **Simulation: the parametric-recovery simulation (generate responses from a fitted DriftIRT or BKT state over six autocorrelated trajectories, refit, compare) is the Sprint 5 item that validates the fitted states; it needs only NumPy.**

8.4 Components implemented and understood (done)

- Data ingest: scripts/ingest_fluid.py downloads the six Fluid Benchmarking CSVs and writes the canonical table. Verified: 12,584,584 rows = 452 checkpoints x 27,842 items, no missing correctness values, correct strictly in {0, 1}, all 57 MMLU subjects present, accuracy rises from first to last

checkpoint for every model (for example OLMo-2 7B 0.24 to 0.66). Time metadata is incomplete: step is missing for Amber and K2 (3.3 million cells) and tokens seen for Amber, K2 and Pythia (7.7 million cells); checkpoint index is complete and is the clock used **by every model so far**.

- Per-skill learning curves: scripts/plot_skill_curves.py produces raw accuracy with Wilson intervals per (model, skill, checkpoint). Findings on OLMo-2 7B (values corrected on September 30, 2026 after independent review): three curve shapes: sharp rise then plateau (world_religions 0.32 to 0.82, high_school_biology 0.18 to 0.73), slow climb (professional_law 0.25 to 0.43), and limited improvement that stays near chance (high_school_mathematics 0.21 to 0.31, peak 0.35). Seven subjects show a peak-minus-final drop strictly above 0.08; **the paired test (Chapter 6.6) has since shown that none of them is a real drop.** Across models on high_school_biology, Pythia-2.8B and Pythia-6.9B decline from 0.32 to about 0.25 over their runs, Amber rises from 0.18 to 0.27, and OLMo-1, OLMo-2 and K2 reach 0.65 to 0.80; **the chance-adjusted check has since shown that the early Pythia and Amber values are answer-letter artefacts.**
- Evaluation harness and baselines: scripts/run_baselines.py runs the trivial predictors under the walk-forward protocol **at horizons 1, 5, 10 and 20**; **fixed validation/test blocks, subject-bootstrap standard errors and paired differences are implemented (tests/test_blocks.py, 14 tests) and every learner-model result in Chapter 6 uses them.**
- **Learner models (all with tests, scored on OLMo-2 MMLU, Chapter 6.5): static IRT persistence; T-SKIRT; per-model recalibration; walk-forward calibration layer; DriftIRT; BKT with per-item emissions and the recency-weighted item mean control. Three of them beat the Workbook 1 bar on held-out checkpoints at every horizon.**
- **Forgetting: the McNemar peak-to-final test with Holm and selection adjustment, and the chance-adjusted check with gold-letter matching (tests/test_forgetting.py, 32 tests). Result: no forgetting candidate survives (Chapter 6.6).**
- **Baseline ladder: ladder tables by model, benchmark and horizon, the per-checkpoint ladder, and the variance decomposition (tests/test_ladder.py, 11 tests; Figure 6.1).**
- Static IRT inputs: Fluid Benchmarking's fitted 2PL item parameters (a, b per item) are loaded, converted to the probit scale where needed, **and recalibrated per model; the recalibration was what made the temporal IRT competitive.**

8.5 Plan for implementing the prototype method

- Sprint 3 (Oct 1 to Oct 14), **done**: clustered standard errors, block bootstrap and fixed validation/test blocks; McNemar paired test with selection control (Aniket); study of pyBKT's E-step and emIRT::dynIRT's data model (Bhagyesh); T-SKIRT with probit-converted item parameters at h = 1 and h = 5 **(done by Ronak; found to lose with borrowed parameters)**; **added: static IRT persistence (Aniket), per-model recalibration, walk-forward calibration, the tau/tau_a/nu sweep and horizons 10 and 20 (Ronak; pulled forward from Sprint 5)**. **Tokens-seen fill for Pythia, Amber and K2 delivered late, on Oct 28 (Aniket; src/llm_kt/time_metadata.py); every model so far was scored with the checkpoint index as its clock.** Deliverable met: first temporal model versus the item-mean bar.
- Sprint 4 (Oct 15 to Oct 28), **done**: log-space batched BKT with per-item guess and slip, 61-setting grid, horizons 5 and 20, mastery figures (Bhagyesh, Oct 27); **added: DriftIRT (replaces the Sprint 5 dynamic IRT) and the baseline-ladder analysis requested by the advisor (Ronak), the chance-adjusted forgetting check (Aniket), the recency-weighted item mean control (Bhagyesh), Decision 6, the Oct 19 advisor meeting and briefing; time-metadata fill and the Phase 2 compute plan (Aniket, Oct 28)**. Not done: DAS3H baseline, parametric-recovery simulation, static IRT diagnosis variant **(carried to Sprints 5 and 6)**. Deliverable exceeded: BKT results at h = 1, 5 **and 20, plus DriftIRT**.
- **Sprint 5 (Oct 29 to Nov 11), revised: adopt the recency-weighted item mean as the bar and rescore T-SKIRT and DriftIRT against it with selection on validation (Ronak); fit every learner model on the other five models to test generalisation beyond OLMo-2 (Bhagyesh); the advisor's held-out transfer test, fitting per-skill trajectories on half the items of each subject and on MMLU only, then scoring the same checkpoints on the held-out items and on ARC and HellaSwag (Aniket); pooled learn/forget rates across models in BKT and the fitted-state artifact contract (Bhagyesh); parametric-recovery simulation at N = 6 (Ronak); subject-level running mean for the ladder caveat; rerun the temporal models with the tokens-seen clock; GitHub Actions CI. The dynamic IRT go/no-go is resolved: DriftIRT is the primary temporal model.**
- Sprint 6 (Nov 12 to Nov 25): bank-level item-drift test with simulation-based power analysis **(Aniket)**; model-based emergence order versus threshold-based **(Bhagyesh, using the BKT mastery curves)**; **DAS3H baseline and** first end-to-end figure set **(Ronak)**; Phase 2 GPU pilot and cost estimate (Aniket).

- Sprint 7 (Nov 26 to Dec 12): semester-1 results write-up and figure set; visualization backend skeleton (FastAPI endpoints over the results and fitted-state Parquet); Phase 2 go/no-go.
- Semester 2 (2027): OLMo-2 on MathCAMPS (or the no-inference fallback), deep KT ablation, GD-GPIRT on subsamples, full visualization, paper drafting, reproducibility package, **conference** submission (**target NeurIPS 2027;** internal completion target end of April; official 2027 deadline to be confirmed, the 2026 paper deadline was May 6).

8.6 Progress log

Date	Item	Status
2026-08-31	First advisor meeting: advisor proposed project ideas; team chose knowledge tracing for LLMs across training checkpoints and refined the framing with her	Done
2026-09-01	Advisor meeting (in person): scope discussed; advisor to share relevant papers	Done
2026-09-02	Project abstract drafted and e-mailed to advisor for review; formatted abstract document built per the course template (Bhagyesh)	Done
2026-09-02	First literature survey with adversarial fact-checking of each recent claim against primary sources; brief "Tracing What LLMs Know"; 115-entry reference list (Bhagyesh)	Done
2026-09-03	Abstract submitted on Canvas; GitHub repository created (Bhagyesh); advisor feedback on abstract received by e-mail (NeurIPS target, add visualization, simplify last paragraph)	Done
2026-09-03	First project plan: research questions and pre-registered hypotheses, baseline tiers, forward-chained held-out next-checkpoint evaluation, item-parameter drift test, simulation study, two-semester timeline; reviewed against the NeurIPS checklist (Bhagyesh)	Done
2026-09-08	Team meeting: advisor feedback reviewed; knowledge-tracing reading divided (BKT; deep KT and IRT; LLM psychometrics and checkpoint releases)	Done
2026-09-15	Team meeting: paper summaries presented; time step = checkpoint raised as Decision 1; two-phase plan (MMLU subjects, then MathCAMPS)	Done
2026-09-22	Systematic literature search (about 25 queries); closest prior work identified	Done
2026-09-23	Team meeting: literature search reviewed, core claim holds, abstract claim to soften; data verification approach agreed	Done
2026-09-26	Abstract presentation slides and speaker script completed (Bhagyesh)	Done

Date	Item	Status
2026-09-29	Design decisions 1 to 5 recorded (time step = checkpoint; model family; data source; skill tags; pseudo-learners)	Done
2026-09-29	Fluid Benchmarking data verified to contain per-item, per-checkpoint correctness	Done
2026-09-29	Abstract v2 drafted per advisor feedback	Done, pending advisor review
2026-09-29	Repository skeleton, ingest pipeline (17 tests), canonical Parquet table	Done
2026-09-29	Per-skill accuracy curves with Wilson intervals; two figures	Done
2026-09-29	Evaluation harness, metrics, five baselines (33 tests); results at h=1 and h=5	Done
2026-09-30	Deep literature review (priorities 1 to 5), spot-verified; model choice updated to dynamic IRT + BKT + T-SKIRT	Done
2026-09-30	Independent review of Workbook 1 and code: corrected empirical numbers (Section 8.4), eight bibliography records, dynamic IRT and BKT feasibility claims, baseline interpretation, Phase 2 scope; harness given a horizon argument and probability checks	Done
2026-09-30	Code, meeting notes, project notes and abstract v2 committed to the shared repository; Bhagyesh's environment set up on a fresh clone (52 tests passing, ruff clean); scripts made executable and docs/ layout aligned with the README	Done
2026-10-01	**Workbook 1 submitted; team briefing written**	**Done**
2026-10-09	**Workbook 1 presentation: speaker notes for twelve slides written in plain language; delivered in class on Oct 9 by Bhagyesh (slides 1 to 6) and Ronak (7 to 12)**	**Done**
2026-10-05 to 10-18	**Sprint 3 code: chronological blocks, subject-bootstrap SEs, paired differences; McNemar forgetting test with selection control; static IRT persistence; T-SKIRT; per-model recalibration; walk-forward calibration; sweeps and horizons 5, 10, 20; 328 tests. Findings: borrowed item parameters lose badly, recalibration gives a tie, T-SKIRT loses at h = 20, forgetting candidates do not survive adjustment**	**Done**
2026-10-19	**Advisor meeting (11:20 to 11:40): results presented from a screen-shared briefing; advisor interested in the baseline ladder; held-out transfer test suggested; chain-of-thought knowledge tracing noted as a later direction; BKT confirmed as next. Sprint 3 code and the Workbook 1 package pushed; ronak_branch and PR workflow started**	**Done**

Date	Item	Status
2026-10-20	**Baseline-ladder analysis with horizons 10 and 20 and variance decomposition; DriftIRT built, selected on validation, beats the bar at h = 1, 5, 20 with leakage audits; chance-adjusted forgetting check with gold-letter matching: all five surviving forgetting pairs are letter artefacts**	**Done**
2026-10-27	**PR #3 merged (ladder, DriftIRT, forgetting check); Decision 6 recorded (PR #4); BKT with per-item emissions, 61-setting grid, horizons 5 and 20, recency-weighted item mean control, mastery figures, design spec and write-up (Bhagyesh, PR #5 merged); team briefing and project notes refreshed with a status table and the chain-of-thought future-work section**	**Done**
2026-10-28	**BKT headline results reproduced on a second machine (agreement to six decimals); tokens-seen clock filled for Pythia, Amber and K2 and the Phase 2 compute plan written (Aniket); Workbook 2 and individual log books written; Gantt and PERT charts drawn**	**Done**
2026-11-11	**Sprint 5: recency-weighted bar adopted and models rescored; five other models; held-out transfer test; recovery simulation; CI**	**Planned**

8.7 Schedule status

Against the Workbook 1 schedule the modelling track is ahead: the Sprint 4 deliverable (BKT results at h = 1 and 5) was met with results at h = 20 as well; the Sprint 5 deliverables (a dynamic IRT result and horizons 10 and 20) were delivered in Sprint 4 through DriftIRT; and three learner models already beat the pre-committed bar on held-out checkpoints, which the plan expected only at the end of the semester. Two secondary items slipped and are carried forward with new dates: the DAS3H baseline (Sprint 6) and the parametric-recovery simulation (Sprint 5). The analysis track delivered the McNemar test, the static IRT baseline and the chance-adjusted check on time and the time-metadata fill three weeks late; the held-out transfer test and the Phase 2 pilot are its two items for Sprints 5 and 6, with the first cost estimate already written. The overall project is ahead of its schedule on results and on schedule on process, with the GPU decision for Phase 2 unchanged at December 12.

Chapter 9
Project Schedule

Two-week sprints. Owners were agreed at the team meeting of September 29, 2026; the split follows three tracks (Ronak: data and infrastructure, evaluation harness, visualization; Bhagyesh: learner models; Aniket: data analyses, statistical tests, Phase 2 evaluation) so that tasks do not overlap. Course assignments are not listed; only project outputs are. Semester-2 dates are in 2027 and assume the standard spring calendar; the conference deadline is unconfirmed and the internal completion target is the end of April. Milestones are in bold in the Deliverable column **of the Workbook 1 schedule; in this revision bold marks changes since October 1: Sprints 3 and 4 are recorded as done with their actual owners and additions, Sprint 5 is rewritten around the new bar and the advisor's held-out test, Sprint 6 gains DAS3H, and the dynamic IRT is replaced by DriftIRT throughout. Figures 9.1 and 9.2 lay the plan out as a Gantt chart and a PERT dependency chart for both semesters.**

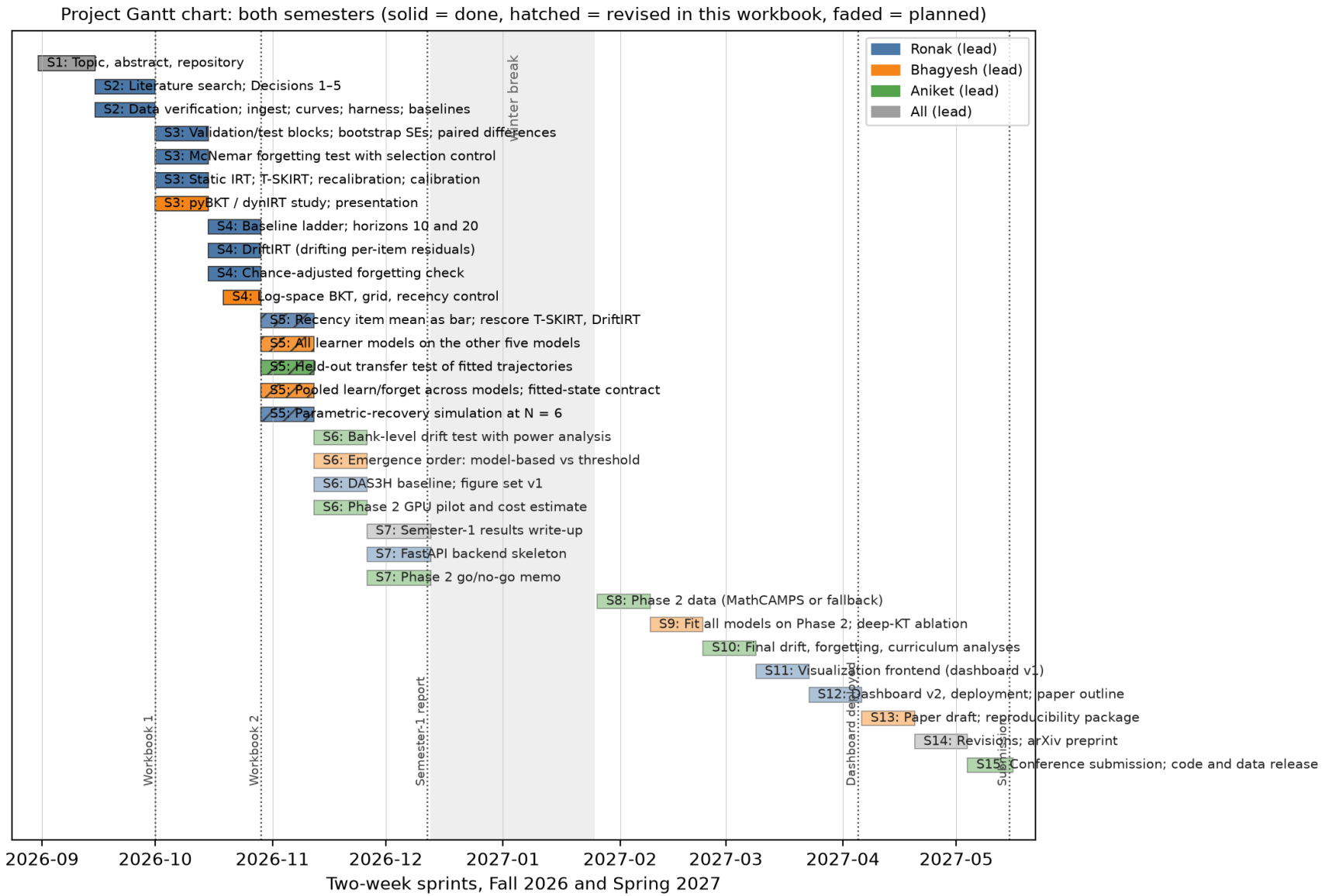


Figure 9.1. Gantt chart for both semesters. Bars are coloured by lead owner; solid bars are done, hatched bars are the Sprint 5 items revised in this workbook, faded bars are planned. Dotted lines mark the course milestones and the submission.

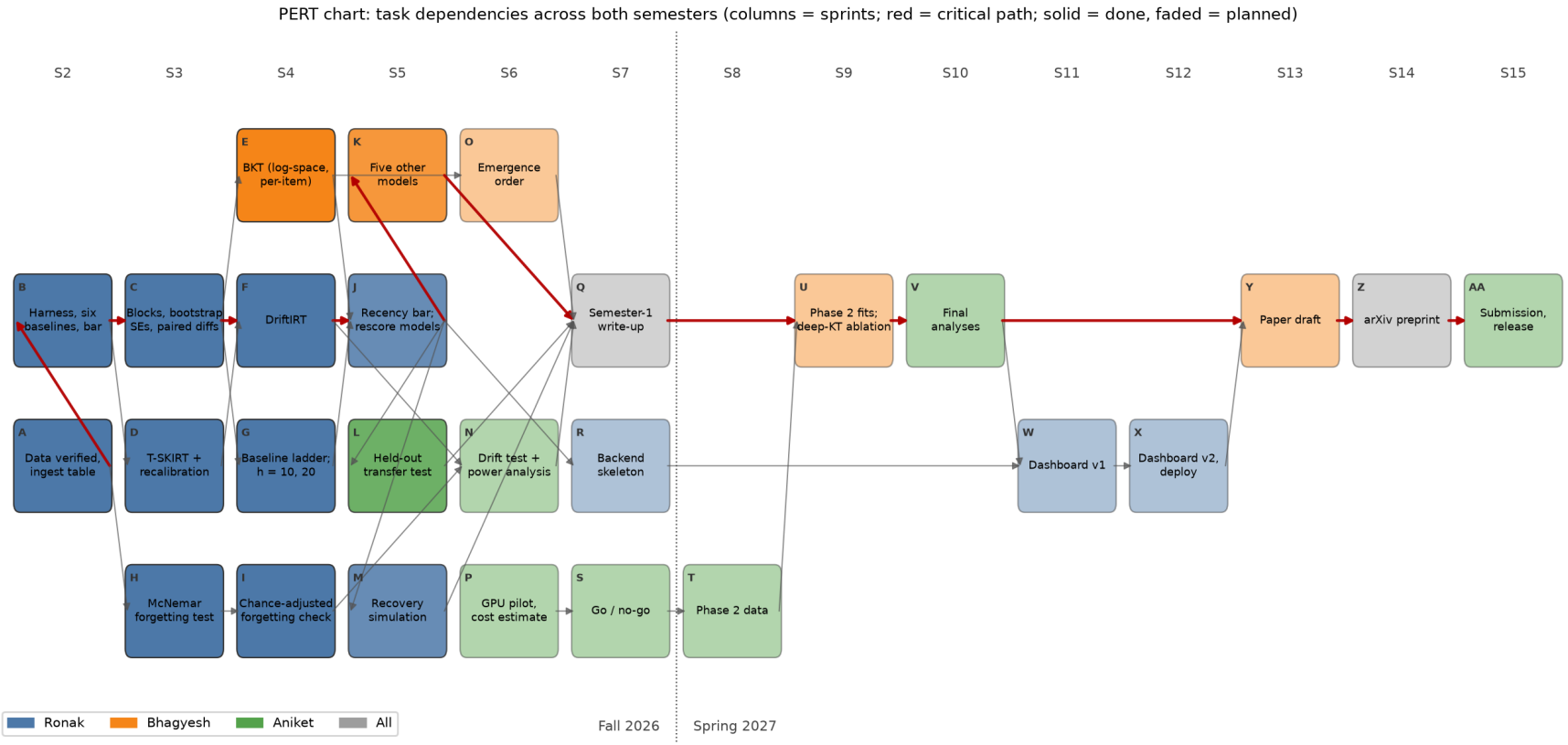


Figure 9.2. PERT chart of task dependencies. Columns are sprints; an arrow means the right-hand task needs the left-hand result. The critical path (red) runs from the data table through the harness, DriftIRT and the new bar to the semester-1 write-up, the Phase 2 fits, the final analyses and the paper. The Phase 2 data branch (GPU pilot, go/no-go, data) joins the critical path at the Phase 2 fits, so a December 12 no-go switches that branch to the fallback datasets without moving the paper dates.

Sprint	Dates	Task	Owner	Deliverable
1	Aug 31 – Sep 14	Topic chosen from advisor's proposed ideas and refined with her (Aug 31, Sep 1); project framing and abstract drafted (Sep 2); first literature survey and project plan (Sep 2–3); repository set up (Sep 3); initial reading	Bhagyesh (background and gap, literature survey, project plan, repository creation); Ronak (method and evaluation, repository structure); Aniket (problem statement)	Project framing agreed; repository live (done)
2	Sep 15 – Sep 30	Literature search and verification; design decisions; data verification; repo, ingest, curves, harness, baselines	Ronak (literature search, data verification, repo, ingest, harness, curves); Bhagyesh (Decisions 1–5 write-up, modeling design); Aniket (release survey, schema draft, meetings folder)	**Canonical dataset; baseline numbers; abstract v2** (done)
3	Oct 1 – Oct 14	**Done.** McNemar paired test with selection control; clustered SEs and block bootstrap; fixed validation/test blocks; pyBKT E-step and dynIRT data-model study; T-SKIRT with probit-converted item parameters; **added: static IRT persistence, per-model recalibration, calibration layer, horizons 10 and 20; presentation**. **Time-metadata fill delivered late, on Oct 28 (Aniket)**	**Ronak (blocks, bootstrap, T-SKIRT, recalibration, calibration); Aniket (McNemar test, static IRT persistence, time-metadata fill); Bhagyesh (pyBKT/ dynIRT study, slides 1–6)**	**First temporal model vs item-mean bar** (done: tie after recalibration)
4	Oct 15 – Oct 28	**Done.** Log-space batched BKT with per-item guess/slip, 61-setting grid, horizons 5 and 20, mastery figures; **added: DriftIRT, baseline-ladder analysis, chance-adjusted forgetting check, recency-weighted item mean control, Decision 6, advisor meeting Oct 19**. Not done: DAS3H (to Sprint 6), recovery simulation and static diagnosis variant (to Sprint 5)	**Bhagyesh (BKT, control, write-up); Ronak (DriftIRT, ladder, briefing, Decision 6); Aniket (chance-adjusted forgetting check, time-metadata fill, Phase 2 compute plan)**	BKT results at h=1, 5 **and 20; DriftIRT results; three models beat the bar** (done)

Sprint	Dates	Task	Owner	Deliverable
5	Oct 29 – Nov 11	**Recency-weighted item mean adopted as the bar; T-SKIRT and DriftIRT rescored against it on validation; all learner models fitted on the other five models; held-out transfer test (fit on half the items per subject and on MMLU, score held-out items and ARC/HellaSwag at the same checkpoints); pooled learn/forget across models; fitted-state artifact contract; parametric-recovery simulation at N = 6; subject-level running mean; rerun of the temporal models with the tokens-seen clock; GitHub Actions CI**	**Ronak (rescoring, recovery simulation, CI, tokens-seen rerun); Bhagyesh (five other models, pooled rates, fitted-state contract); Aniket (held-out transfer test)**	**Generalisation and transfer results; validated fitted states**
6	Nov 12 – Nov 25	Bank-level drift test with power analysis; emergence order comparison **(from BKT mastery curves)**; **DAS3H baseline;** figure set; Phase 2 GPU pilot and cost estimate	Aniket (drift test, GPU pilot); Bhagyesh (emergence analysis); Ronak (**DAS3H,** figures)	Analysis figures v1
7	Nov 26 – Dec 12	Semester-1 results write-up and figure set; FastAPI backend skeleton consuming the Sprint 5 fitted-state contract (Ronak builds; Bhagyesh takes over data endpoints in Sprint 11); Phase 2 go/no-go	Bhagyesh (model results sections); Aniket (analysis sections, go/no-go memo); Ronak (backend, figures)	**Semester-1 results write-up; backend skeleton**
8	Jan 26 – Feb 8, 2027	Phase 2 data: OLMo-2 on MathCAMPS, or DataDecide/ ElementalTask fallback with ID and coverage validation	Aniket (eval runs or fallback validation); Ronak (ingest extension); Bhagyesh (skill mapping)	Phase 2 canonical table
9	Feb 9 – Feb 22, 2027	Fit all models on Phase 2 data; deep KT ablation with leave-one-model-out on Phase 1 data (and on Phase 2 if it has more than one model); static skill-profile comparison	Bhagyesh (models); Aniket (ablation, static profiles); Ronak (harness)	Phase 2 results
10	Feb 23 – Mar 8, 2027	GD-GPIRT on subsamples; final drift and forgetting analyses **(with the chance-adjusted check)**; curriculum-order comparison	Aniket (GD-GPIRT, forgetting); Bhagyesh (drift, curriculum order); Ronak (results pipeline)	**Final analysis set**

Sprint	Dates	Task	Owner	Deliverable
11	Mar 9 – Mar 22, 2027	Visualization frontend: mastery curves with filtered/smoothed states, heatmap, forgetting markers, drift view, model comparison	Ronak (frontend); Bhagyesh (data endpoints); Aniket (user testing with advisor)	Dashboard v1
12	Mar 23 – Apr 5, 2027	Dashboard v2 and deployment; paper outline and related work	Ronak (deployment); Bhagyesh (method and results outline); Aniket (related work, evaluation section)	**Dashboard deployed**
13	Apr 6 – Apr 19, 2027	Paper draft; internal review; reproducibility package (pinned versions, run manifest, data-building scripts)	Bhagyesh (method, results); Aniket (introduction, related work, discussion); Ronak (figures, reproducibility package)	Paper draft v1
14	Apr 20 – May 3, 2027	Revisions from internal and advisor review; arXiv preprint	Bhagyesh (revisions to method/results); Aniket (revisions to framing); Ronak (camera-ready figures, arXiv submission)	**arXiv preprint**
15	May 4 – May 15, 2027	**Conference** submission (date to be confirmed against the 2027 call); code and data release	Aniket (submission and checklist); Ronak (code/data release); Bhagyesh (supplementary material)	**Conference submission; public release**

9.1 Individual contributions to date

- Ronak Patel: abstract paragraph 3 (method, data, evaluation) and the abstract v2 rewrite; reading and summaries on deep knowledge tracing and its link to IRT; the systematic literature search (about 25 queries) and the deep-review brief; verification of the Fluid Benchmarking release; repository skeleton and packaging; ingest pipeline and canonical table; evaluation harness, metrics and five baselines with tests; per-skill curve figures; design-decision and project notes; Workbook 1 assembly and the corrections from its independent review. **Since October 1: chronological blocks, subject-bootstrap standard errors and paired differences; T-SKIRT, per-model recalibration, the walk-forward calibration layer and DriftIRT, with their sweeps, horizon studies**

and leakage audits; the baseline-ladder analysis; speaker notes for the presentation and delivery of slides 7 to 12; the October 19 advisor briefing and meeting note; two team briefings; Decision 6; the pull-request workflow (PRs #1 to #4); reproduction of the BKT results; Workbook 2, the Gantt and PERT charts, and the individual log books.

- Bhagyesh Rathi: created the GitHub repository (initial commit, licence and ignore rules) and the shared artifacts folder for course submissions; prepared the scope questions and the request for reviewed papers for the 2026-09-01 advisor meeting; abstract paragraphs 1–2 (background and gap statement) and the formatted abstract submission (title page and body per the course template); abstract presentation slides and speaker script; an early literature survey (September 2–3) over arXiv, OpenReview and recent venue proceedings in which every recent claim was independently fact-checked against primary sources, producing the brief "Tracing What LLMs Know", a 115-entry reference list, and a first project plan with pre-registered research questions, baseline tiers, a forward-chained held-out next-checkpoint evaluation, an item-parameter drift test, a simulation study and a two-semester timeline, reviewed against the NeurIPS checklist; reading and summaries on classic and extended Bayesian Knowledge Tracing (Corbett & Anderson; Khajah, Lindsey & Mozer; pyBKT); write-up of Design Decisions 1 and 2 (time step = checkpoint; model family given few learners) with pros and cons, and preparation of Decisions 3–5 for team adoption; note-taker for the 2026-09-01 and 2026-09-29 meetings; advisor liaison, including the request for a recurring meeting slot; environment set up and the test suite verified on a fresh clone (52 tests) on 2026-09-30. **Since October 1: study of pyBKT's E-step and the emIRT::dynIRT data model; delivery of presentation slides 1 to 6; arranging the October 19 advisor meeting; the BKT implementation (log-space forward-backward with batched per-item emissions, MAP-EM with shrunk recency-weighted per-item guess and slip, skill and Fluid anchors, mastery curves; 26 tests), the recency-weighted item mean and skill-mean baselines, the BKT options in run_models.py, the 61-setting grid and the horizon 5 and 20 runs, scripts/analyze_bkt.py with its tables and four figures, the BKT design spec and plan, the BKT write-up in the project notes and team briefing, and the chain-of-thought future-work section (PR #5, 14 commits, merged October 27); the README learner-model section and results table, on bhagyesh_branch awaiting its pull request.**
- Aniket Sarrin: one-paragraph problem statement that seeded the abstract; reading and summaries on psychometrics for LLMs and public checkpoint releases (Fluid Benchmarking, MathCAMPS, Pythia, OLMo); survey of which

checkpoint releases publish item-level results; draft of the repository structure and canonical data schema; creation of the meetings folder and template; **note-taker for the September 8 and 15 team meetings. Since October 1: the McNemar peak-to-final forgetting test with Holm and selection adjustment (src/llm_kt/forgetting.py, scripts/run_forgetting.py); the static IRT persistence baseline with Fluid's 2PL parameters (src/llm_kt/static_irt.py); the chance-adjusted forgetting check with gold-letter matching, per-letter accuracy, G-test and residual drop, and the verdict that no forgetting candidate survives (32 tests); the tokens-seen clock for Pythia, Amber and K2 (src/llm_kt/time_metadata.py); the Phase 2 compute plan with GPU options and a first cost estimate (docs/phase2-gpu-plan.md); owner of the Phase 2 pilot (November 25), the go/no-go memo (December 12) and the advisor's held-out transfer test (Sprint 5).**